scoup.ai

scoup.ai architecture, decision records, innovation thesis and dashboard spec

2026-10-18

> **Pre-product. Synthetic data only. Not investment advice.**

Documents in this volume:

- docs/architecture/overview.md
- docs/architecture/data-model.md
- docs/architecture/api-surface.md
- docs/architecture/dev-portal-drift.md
- docs/adr/README.md
- docs/adr/0001-typescript-platform-fastify-prisma-postgres.md
- docs/adr/0002-rls-fail-closed-tenancy.md
- docs/adr/0003-python-core-kept-as-reference.md
- docs/adr/0004-hash-chained-append-only-audit.md
- docs/adr/0005-four-eyes-separation-of-duties.md
- docs/adr/0006-scoped-tenant-api-keys.md
- docs/adr/0007-human-approval-gate-content-hash.md
- docs/adr/0008-deterministic-engines-no-runtime-model.md
- docs/adr/0009-sandboxed-document-parsing.md
- docs/adr/0010-single-rbac-matrix-declared-route-access.md
- docs/adr/0011-money-as-integer-cents.md
- docs/adr/0012-aws-target-infrastructure-not-applied.md
- docs/innovation-thesis.md
- docs/exec-dashboard-spec.md

Generated from source by tools/pdf/build.py at source commit 3b18982d0662. Figures are drawn from source data at build time.

Contents

scoup.ai architecture overview (as built) 4
- 1. System context 4
- 2. Module map 5
- 3. Dependency direction 7
- 4. Tenancy and row-level security 7
- 5. Authentication and authorisation 7
- 6. Trust boundaries 8
- 7. Audit trail 8
- 8. What is not verified 9

Data model 10
- 1. Conventions that apply everywhere 10
- 2. Entity-relationship view (core claim flow) 10
- 3. Tables by group 11
- 4. Status machines 13
- 5. What the schema does not hold 14

API surface (TypeScript API) 15
- 1. Summary 15
- 2. Conventions every route shares 15
- 3. Route inventory 16

Developer portal drift 21
- 1. The short version 21
- 2. What the portal documents that the TS API lacks 21
- 3. What the TS API has that the portal does not mention 22
- 4. Recommendation 22
- 5. Open questions 23

Architecture decision records 24

ADR 0001: TypeScript platform on Fastify 5, Prisma 7 and PostgreSQL 16 25
- Context 25
- Decision 25
- Consequences 25
- Alternatives 25

ADR 0002: Tenant isolation in three fail-closed layers, anchored in PostgreSQL RLS 26
- Context 26
- Decision 26
- Consequences 26
- Alternatives 26

ADR 0003: The Python core stays as the reference implementation and parity oracle 27
- Context 27
- Decision 27
- Consequences 27
- Alternatives 27

ADR 0004: Hash-chained, append-only audit trail written in the business transaction 28
- Context 28
- Decision 28
- Consequences 28

ADR 0005: Four-eyes separation of duties on claim and document decisions 29
- Context 29
- Decision 29
- Consequences 29

ADR 0006: Scoped tenant API keys bound to their creator's current role 30
Context 30
Decision 30
Consequences 30

ADR 0007: A human approval gate bound to a content hash; the product sends nothing 31
Context 31
Decision 31
Consequences 31

ADR 0008: Deterministic, explainable engines; no trained model or LLM call at runtime 32
Context 32
Decision 32
Consequences 32

ADR 0009: Untrusted documents are parsed in a bounded child process 33
Context 33
Decision 33
Consequences 33

ADR 0010: One frozen RBAC matrix in the shared package; every route declares its access 34
Context 34
Decision 34
Consequences 34

ADR 0011: Money is stored as integer cents and computed in exact decimal arithmetic 35
Context 35
Decision 35
Consequences 35

ADR 0012: AWS target infrastructure as Terraform, validated but never applied 36
Context 36
Decision 36
Consequences 36

Innovation thesis 37
1. Summary 37
2. I1. Accessorial and surcharge audit against the shipper's own contract terms 38
3. I2. A regulation-coded ocean D&D audit for SME importers 40
4. I3. One claims queue for a shipper's truck and ocean charges 41
5. Later segment: carriers and brokers 42
6. Hypotheses without outside demand evidence 45
7. What we will not claim 45

Executive dashboard: specification input 46
1. Why this page exists 46
2. Audience and access 46
3. Definitions 46
4. Metric catalogue 47
5. Routes and implementation constraints 50
6. Gaps: what executives will ask for that the data model cannot answer 50
7. Acceptance criteria 51
8. Open questions for the owner 53

scoup.ai architecture overview (as built)

> **Status.** Pre-product. Synthetic data only, never deployed, no paying customers. Everything below describes code on branch `feat/hardening` of the TypeScript monorepo (`ventures/freight-recovery-build`, head `ed3d126`, read on 2026-10-17). Where a control has only been verified locally, we say so. Nothing in this document is a claim about accuracy, security in production or recovery rates; none of those has been measured.

scoup.ai (repository name `freight-recovery`) assembles evidence packets and draft demand letters for freight billing disputes: truck detention and accessorial overcharges, fuel-surcharge errors and ocean demurrage and detention (D&D) invoices. A person always reviews and approves a packet before anything counts as send-ready, and the product never sends a demand or moves money itself.

This document is the map. The detailed, step-by-step design record is the 4,099-line `ARCHITECTURE.md` in the build worktree, and the threat model is `technical/web-platform-security.md`. We cite both by section rather than repeat them.

1. System context

The system has two codebases that live side by side in one repository. The owner decided that the Python reference core stays and that everything customer-facing is TypeScript (see ADR 0003).

Part	Path	Language and stack	Role today
Web API	`api/`	Node 22, Fastify 5.12, Prisma 7.10, PostgreSQL 16, Zod 4, Pino	The product backend: auth, tenancy, imports, analysis, approvals, exports, intelligence, ocean D&D
Web app	`web/`	React 19.3, Vite, React Router 7, TanStack Query 5, Zustand 5	The single-page app customers would use
Shared contract	`packages/shared/` (`@fr/shared`)	TypeScript, Zod only	RBAC matrix, DTO schemas, canonical JSON, and the pure rule engines (analysis, charge-intel, fuel, outlook, ocean-dd)
Python reference core	`src/freight_recovery/`	Python 3.11+, FastAPI, SQLAlchemy 2, Alembic	Reference implementation of ingest, extraction, rules and evidence; the TS port is checked against it for exact parity
Pilot dashboard	`webapp/`	Vite + React	Reference only, frozen
Infrastructure	`infra/*.tf`	Terraform (AWS)	`fmt` and `validate` only, never applied
Developer portal	`developer-portal/`	MkDocs Material	Documents the **Python** API only (see dev-portal-drift.md)

Diagram source (Mermaid), shown as text; it is not rendered in this PDF.

```
%%{init: {'theme': 'neutral'}}%%
graph LR
    U["Browser (customer staff)"] --> CF["CloudFront"]
    M["Machine client (tenant API key)"] --> CF
    CF --> SPA["web/ SPA (S3)"]
    CF --> ALB["ALB + WAFv2"]
    ALB --> API["api/ Fastify on ECS Fargate"]
    API --> PG["PostgreSQL 16 (RLS)"]
    API --> R["Redis 7 (rate-limit store)"]
    API --> S3["S3 documents (SSE-KMS)"]
    API --> W["Parse worker child process"]
```

```
    API -. "imports engines" .-> SH["@fr/shared"]
    SPA -. "imports contract" .-> SH
    PY["src/freight_recovery (Python)"] -. "parity tests only" .-> SH
```

The AWS side of the diagram is the Terraform design in `infra/`; it has never been applied. The local stack (`compose.web.yml`) uses MinIO, `postgres:16.15-alpine` and `redis:7-alpine` in its place.

2. Module map

2.1 API (`api/src/`)

Each feature folder registers its routes in `api/src/app.ts` (`buildApp`). Every route goes through `defineRoute` in `api/src/http/route.ts`, which refuses to boot the app when a route has no access declaration. The full route list is in api-surface.md.

Folder	Responsibility	Key files
`http/`	One `onRequest` security pipeline (rate limit, Origin, CSRF, authN, permission), route declaration, error mapping, DB-guard to HTTP mapping, client address behind CloudFront	`security.ts`, `route.ts`, `errors.ts`, `db-guard.ts`, `client-ip.ts`, `strict-body.ts`
`security/`	argon2id passwords, HS256 JWT with per-purpose keys, TOTP, CSRF tokens, HKDF/AES-GCM, lockout math, password policy	`password.ts`, `jwt.ts`, `totp.ts`, `csrf.ts`, `crypto.ts`, `lockout.ts`
`auth/`	Login, MFA, refresh rotation with reuse detection, recovery, invites, user management, API-key authentication, dev mail outbox	`routes.ts`, `service.ts`, `session.ts`, `users-routes.ts`, `api-key-auth.ts`
`apikeys/`	Tenant API keys: create, list, revoke; key material and HMAC verification	`routes.ts`, `service.ts`, `key-material.ts`
`claims/`	Claim list and detail, packet viewer, review state machine, approve/reject/send, analyze (single and bulk), content hash	`routes.ts`, `service.ts`, `state-machine.ts`, `content-hash.ts`, `analysis.ts`
`imports/`	Upload pipeline, MIME sniffing, sandboxed parsing, review queue, commit to claims	`routes.ts`, `service.ts`, `review-service.ts`, `commit-service.ts`, `sandbox/*`, `parse/*`
`exports/`	Streaming CSV/XLSX exports of claims, packets and outcomes with formula neutralisation	`routes.ts`, `rows.ts`, `writers.ts`, `neutralize.ts`
`intelligence/`	Worklist (`priority-v1`), similar claims, provenance, charge-line suggestions and team labels, fuel tables and fuel check, cost outlook	`routes.ts`, `service.ts`, `priority.ts`, `charge-routes.ts`, `fuel-routes.ts`, `outlook-routes.ts`
`ocean-dd/`	D&D tariffs, terminal calendars, invoices and the audit run that creates an `OCEAN_DD` claim with packet revision 1	`routes.ts`, `service.ts`, `audit-run.ts`
`audit/`	Hash-chained audit log, append under advisory lock, verifier, four-eyes author readers	`audit.ts`, `chain.ts`, `routes.ts`, `label-authors.ts`, `dd-editors.ts`
`platform/`	Health, platform dashboard (pipeline stats, telemetry, logs, flags, eval runs, platform audit), SUPER_ADMIN cross-tenant read, price store and importer CLI	`routes.ts`, `dashboard-routes.ts`, `cross-tenant.ts`, `flags.ts`, `price-store.ts`, `prices-cli.ts`
`db/`	Tenant client (fail-closed Prisma extension), system-mode transactions, advisory-lock helpers	`tenant.ts`, `system.ts`, `client.ts`, `*-locks.ts`
`eval/`	Offline evaluation CLIs (`eval`, `eval:charges`, `eval:outlook`) that gate CI on synthetic fixtures	`cli.ts`, `runner.ts`, `charges.ts`, `outlook.ts`
`observability/`, `logging.ts`	Metrics registry, in-memory log ring buffer, Pino logger with key scrubbing	`metrics.ts`, `ring-buffer.ts`
`storage/`	S3 client with tenant-prefix checks on every call	`s3.ts`

2.2 Shared contract (`packages/shared/src/`)

`@fr/shared` has one runtime dependency (Zod). It holds two kinds of code. The first is the contract both sides import: `rbac.ts` (roles, the frozen permission matrix, `can`), `dto.ts` and the per-feature `*-dto.ts` schemas, `audit-actions.ts`, `feature-flags.ts`, `canonical-json.ts`, `money.ts`, `decimal.ts`, `text-sanitize.ts`, `honest-copy.ts` and `transparency-copy.ts`. The second is the set of pure, deterministic engines: `analysis/` (the TS port of the Python rules, letter and Markdown), `charge-intel/` (the `charge-knn-v1` classifier and seed set), `fuel/`, `outlook/` and `ocean-dd/` (free time, seven audit rules, CSV parser, letter, 46 CFR text). Engines take values and return values. They never touch the database or the network, and that property is what makes the parity tests and the eval gates possible.

2.3 Web app (`web/src/`)

`app-root.tsx` lazy-loads one page per feature. `api/client.ts` keeps the access token in memory only, adds the CSRF header and runs a single-flight refresh across tabs. The web app imports the RBAC matrix only to hide controls; the API enforces every permission itself.

Route	Feature folder	Page
`/login`, `/login/mfa`, `/login/mfa-setup`, `/forgot`, `/reset`, `/invite`	`features/auth`	Sign-in, MFA, recovery, invite acceptance
`/claims`, `/claims/:id`	`features/claims`	Claims table and claim sheet (packet, analyze, documents, intelligence tabs)
`/approvals`	`features/approvals`	Human approval queue
`/import`, `/import/review`	`features/imports`	Upload and document review queue
`/intelligence`	`features/intelligence`	Prioritised worklist
`/outlook`	`features/outlook`	Cost outlook from public price data
`/ocean`	`features/ocean-dd`	D&D invoices, tariffs, calendars, CSV import
`/settings/fuel-tables`, `/settings/api-keys`	`features/fuel`, `features/apikeys`	Tenant settings
`/dev/*`	`features/dev`	Platform dashboard (pipeline, telemetry, logs, flags, evaluation, audit)

There is no executive or summary page today. ../exec-dashboard-spec.md specifies one.

2.4 Python reference core (`src/freight_recovery/`)

The Python service is a complete small product of its own: `ingest/`, `extraction/` (a deterministic `Key: Value` stub behind an `ExtractionProvider` interface), `rules/` (detention and invoice checks in `Decimal`), `evidence/` (packet and draft letter), `sandbox/`, `db/`, `storage/`, `api/main.py` and `admin.py`. In the TS platform it has one job: it is the oracle for parity. `api/test/parity/rules.parity.test.ts` and `extract.parity.test.ts` run `tools/parity/*.py` and require exact string equality with the TS engines, apart from the registered divergences (D1 to D8 for extraction, RD1 to RD13 for rules) listed in `ARCHITECTURE.md`. In CI a missing Python interpreter fails the suite.

2.5 Infrastructure (`infra/`)

The Terraform files describe CloudFront with a response-headers policy, an S3 bucket for the SPA (origin access control), an ALB reachable only from CloudFront with an origin-verify header, WAFv2, ECS Fargate in private subnets with VPC endpoints and no NAT, RDS PostgreSQL 16 with `rds.force_ssl=1`, ElastiCache Redis

7 with TLS, an S3 documents bucket with SSE-KMS and a task role limited to the `t/*` prefix, KMS keys and seven Secrets Manager containers whose values are set out of band. Migrations run as a one-off ECS task. Nothing has been applied, and CI runs `terraform fmt -check` and `validate` only.

3. Dependency direction

Dependencies point one way, and lint rules plus architecture tests keep them that way.

```
web/  ----->  @fr/shared  <----  api/
                                   |
                                   +--> PostgreSQL  (through db/tenant.ts or db/system.ts only)
                                   +--> S3          (through storage/s3.ts only)
                                   +--> parse worker (child process, imports/sandbox/)
src/freight_recovery (Python): called by parity tests only, never at runtime
```

- `@fr/shared` imports nothing from `api/` or `web/`, and its engines import nothing that does I/O.
- Feature folders in `api/src` reach the database only through a `TenantDb` handle from `db/tenant.ts`. `db/system.ts` (system-mode transactions) is restricted by lint to auth bootstrap, the evaluation recorder, the price importer and the platform statistics function.
- Platform modules may not import tenant models or tenant tables and may not read `email` (architecture tests, threat model T38).
- The API never calls the Python service at runtime. The TS engines are the production path.

4. Tenancy and row-level security

Tenancy is three layers, and each is built to fail closed on its own. The reasoning is in ADR 0002.

1. **PostgreSQL RLS.** Every tenant table has `ENABLE` and `FORCE ROW LEVEL SECURITY` with `USING/WITH CHECK (tenant_id = fr_current_tenant())`. With no `app.tenant_id` set in the transaction, a query sees zero rows. Across the 11 migrations, 26 tables carry `FORCE ROW LEVEL SECURITY`: the 20 tenant tables (of which `memberships` and `invites` also admit system mode for the auth bootstrap), `audit_events`, `feature_flags`, the two eval tables and the two global price tables. The API connects as `freight_app` (`NOSUPERUSER NOBYPASSRLS NOINHERIT`, not the owner, DML grants only, column-level grants on packets and flags). `freight_owner` owns the schema and runs migrations.
2. **Fail-closed Prisma extension** (`api/src/db/tenant.ts`). Only the 20 models in `TENANT_MODELS` are reachable. The extension injects `tenantId` into every `where` and every `create`, throws on a conflicting value and removes raw SQL from the tenant client. A model added later throws `TenantScopeError` until someone classifies it.
3. **Request context.** User, role, tenant status and refresh-family liveness are reloaded from the database on every request (`auth/session.ts`), never taken from token claims, headers or the body. Platform users get `db = null`.

Composite foreign keys (`[id, tenantId]` references on every child table) stop a row from pointing at another tenant's parent even if application code tries. Object storage repeats the check: a CHECK forces `import_documents.storage_key` to start with `t/<own tenant_id>/`, and every `ObjectStore` call re-checks the prefix. Another tenant's id always answers 404.

5. Authentication and authorisation

Two kinds of caller exist, and the `Authorization` header alone decides which path a request takes (`api/src/http/security.ts`).

Caller	Credential	Path
Person in the web app	Access JWT (10 min, HS256, held in JS memory) plus the `fr_rt` refresh cookie (`HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth`)	CSRF double-submit plus Origin check on unsafe methods; TOTP MFA mandatory for OWNER, ADMIN, PLATFORM_DEV and SUPER_ADMIN; refresh rotation with family revocation on reuse; lockout of `min(30 x 2^(f-5), 900)` seconds after 5 failures
Machine client	`Authorization: Bearer fr_live_<keyId>_<secret>`	Per-IP failure gate, constant-time HMAC compare, revocation and expiry read on every request, route opt-in, scope check, then a re-check that the key creator's **current** role still holds the scope's permission (ADR 0006)

Authorisation is one frozen matrix in `packages/shared/src/rbac.ts`: six tenant roles (OWNER, ADMIN, MANAGER, REVIEWER, ANALYST, VIEWER), two platform roles (PLATFORM_DEV, SUPER_ADMIN) and 30 permissions. Platform roles hold no tenant data permission. An API key carries at most three scopes (`claims.read`, `exports.claims`, `imports.write`), and 9 of the 97 routes accept keys. AuthN and authZ run in `onRequest`, before the body is parsed, so a forbidden caller gets 403 even when the body is invalid.

Two kinds of decision need a second person (ADR 0005). Anyone who authored any packet revision of a claim, a team charge label that influenced its analysis, or the ocean D&D invoice behind it cannot approve or reject that claim. The uploader or a field editor of an imported document cannot accept it. The service check is the primary control, and database triggers back it up.

6. Trust boundaries

#	Boundary	What crosses it	Control at the boundary
B1	Internet to CloudFront and ALB	All HTTP traffic	WAFv2 managed rules and size limits, ALB reachable only from CloudFront with an origin-verify header, TLS 1.2/1.3 (IaC only, unverified in AWS)
B2	HTTP to handler	Requests from browsers and machine clients	`http/security.ts` pipeline; Zod validation in `preHandler`; per-route body limits; strict outgoing schemas on platform routes
B3	API process to parse worker	Untrusted document bytes	One child process per job with wall-clock, memory, output, page and pixel limits and a clean environment (`imports/sandbox/`). This is process isolation, not an OS sandbox
B4	API to PostgreSQL	All tenant data	`freight_app` without BYPASSRLS, RLS with FORCE, the tenant extension, and triggers for state machines, append-only tables and four-eyes
B5	API to S3	Original documents and extracted text	Tenant prefix `t/<tenant_id>/`, SSE-KMS required by bucket policy, task role scoped to `t/*`
B6	Tenant data to platform staff	Operational data	Platform roles have no tenant DB handle; only `fr_platform_pipeline_stats` crosses tenants and it returns counts and durations; a SUPER_ADMIN cross-tenant read is read-only, needs a reason and is audited into the target tenant's chain first
B7	Product to the outside world	Demand letters, money	Nothing crosses today. "Send" only marks a packet send-ready after a content-hash match; no email, portal submission or payment exists

7. Audit trail

Every mutating handler appends to a per-tenant hash chain in the same transaction as the business change (`api/src/audit/audit.ts`): `hash_n = SHA-256(hash_{n-1} + "\n" + canonicalJson(event_n))`, under `pg_advisory_xact_lock`. If the append fails, the request returns 500 and nothing changes. Triggers block UPDATE, DELETE and TRUNCATE on `audit_events`, `approvals`, `login_attempts` and the packet child tables. `GET /api/v1/audit/verify` recomputes the chain. The threat model (T10) records the residual risk: nothing anchors chain heads outside the database yet.

8. What is not verified

The hardening notes in `ARCHITECTURE.md` list what ran: unit, integration and acceptance suites on Node 22.23.3, locally on Windows, against fresh PostgreSQL 16.15 and MinIO clusters. GitHub CI for the latest commits, local Terraform `fmt`/`validate`, container images, the compose stack, Playwright end-to-end tests and any AWS deployment remain unverified. No external penetration test or compliance review has taken place.

Data model

> **Status.** Pre-product, synthetic data only, never deployed. Source: `api/prisma/schema.prisma` (1,002 lines) and the 11 SQL migrations in `api/prisma/migrations/` on branch `feat/hardening` (`ed3d126`). Prisma describes columns; the SQL migrations add what Prisma cannot express (RLS policies, grants, CHECKs, triggers, `SECURITY DEFINER` functions). When the two disagree, the migrations are what the database enforces.

The schema has 35 models in 7 groups. Money is always an integer number of US cents (`Int`, column suffix `_cents`), never a float. Timestamps are `timestamptz(3)`. Primary keys are UUIDs. Every tenant child table references its parent through a composite key `[id, tenantId]`, so a row cannot point at a parent in another tenant.

1. Conventions that apply everywhere

Convention	Where it is enforced	Effect
`tenant_id` on every tenant row	Prisma extension (`api/src/db/tenant.ts`) and RLS policy `tenant_id = fr_current_tenant()`	Rows outside the transaction's tenant are invisible and unwritable
Composite foreign keys `[x_id, tenant_id] -> [id, tenant_id]`	Prisma `@relation(fields: [..., tenantId])` plus `@@unique([id, tenantId])`	Cross-tenant references are impossible at the database level
Append-only tables	`fr_append_only()` triggers on UPDATE, DELETE and TRUNCATE	History cannot be edited, even by the schema owner outside the dev-only reset path
Guarded mutable tables	`*_update_guard` triggers	Only named columns may change, along defined transitions
`version` integer on editable configuration	Service compares it; some triggers require `version + 1`	Optimistic concurrency: a stale write answers 409
`created_by_id` / `updated_by_id`	Service writes them from the request context	Feeds four-eyes author checks

2. Entity-relationship view (core claim flow)

Diagram source (Mermaid), shown as text; it is not rendered in this PDF.

```
%%{init: {'theme': 'neutral'}}%%
erDiagram
    TENANT ||--o{ MEMBERSHIP : has
    USER ||--o| MEMBERSHIP : "belongs to one tenant"
    TENANT ||--o{ CLAIM : owns
    CLAIM ||--o{ EVIDENCE_PACKET : revisions
    EVIDENCE_PACKET ||--o{ PACKET_FINDING : contains
    EVIDENCE_PACKET ||--o{ PACKET_SOURCE : cites
    EVIDENCE_PACKET ||--o{ PACKET_TIMELINE_EVENT : orders
    CLAIM ||--o{ APPROVAL : decisions
    EVIDENCE_PACKET ||--o{ APPROVAL : "decided on"
    IMPORT_BATCH ||--o{ IMPORT_DOCUMENT : groups
    IMPORT_DOCUMENT ||--o{ EXTRACTED_FIELD : yields
    IMPORT_DOCUMENT ||--o{ IMPORT_REVIEW_DECISION : reviewed
    CLAIM ||--o{ CLAIM_DOCUMENT : links
    IMPORT_DOCUMENT ||--o| CLAIM_DOCUMENT : "linked once"
    CLAIM ||--o| CLAIM_FUEL_INPUT : "fuel inputs"
```

```
    FSC_TABLE ||--o{ CLAIM_FUEL_INPUT : "used by"
    CLAIM ||--o| DD_INVOICE : "OCEAN_DD source"
    DD_TARIFF ||--o{ DD_INVOICE : prices
    DD_CALENDAR ||--o{ DD_INVOICE : calendar
```

3. Tables by group

3.1 Identity and access

Table (model)	Tenant-scoped	Purpose	Notable fields and rules
`tenants` (`Tenant`)	is the tenant	Customer organisation	`slug` unique, `status` ACTIVE/SUSPENDED; read-only through the tenant client
`users` (`User`)	no (global)	Person account	`email` unique, `password_hash` (argon2id), `status`, optional `platform_role`, `password_changed_at`
`memberships` (`Membership`)	yes, plus system mode	A user's single tenant and role	`user_id` unique (one tenant per user); `role` OWNER to VIEWER
`invites` (`Invite`)	yes, plus system mode	Pending invitation	`token_hash` unique (raw token never stored), `expires_at`, `accepted_at`, `revoked_at`
`refresh_tokens` (`RefreshToken`)	no	Rotating refresh tokens	`token_hash` (HMAC), `family_id`, `used_at` (reuse revokes the family), `family_expires_at` (30-day cap)
`mfa_secrets` (`MfaSecret`)	no	TOTP secret	AES-GCM `ciphertext`, `nonce`, `key_id`; `last_used_step` blocks replay
`mfa_recovery_codes` (`MfaRecoveryCode`)	no	One-time recovery codes	`code_hash`, `used_at`
`password_resets` (`PasswordReset`)	no	Reset tokens	`token_hash`, `expires_at`, `used_at`
`login_attempts` (`LoginAttempt`)	no	Login outcome log	keyed by `email_hash`; append-only
`login_lockouts` (`LoginLockout`)	no	Failure counter	per `email_hash`, `locked_until`
`mail_outbox` (`MailOutbox`)	no	Dev and test mail transport	refused as a transport in production

3.2 Claims, packets and decisions

Table	Purpose	Notable fields and rules
`claims` (`Claim`)	One dispute case	`claim_number` unique per tenant; `kind` FREIGHT/OCEAN_DD (trigger `claims_kind_guard` blocks changes); `perspective` SHIPPER/CARRIER; `status` (trigger `claims_status_guard`); `amount_claimed_cents`, `recoverable_cents`, `pending_review_cents`; `carrier_name`, `shipper_name`, `load_number`, `invoice_number`, `invoice_date`; `assignee_id`; `version`; `created_at`, `updated_at`
`evidence_packets` (`EvidencePacket`)	One revision of a claim's packet	`revision` unique per claim; `status` adds SUPERSEDED; `demand_letter`, `disclaimer`; `recoverable_cents`, `pending_review_cents`; `content_hash`; `verifier_status` NOT_RUN/PASSED/FAILED/NEEDS_REVIEW; `created_by_id`; `created_at`. Only `status` may change (`evidence_packets_update_guard`); an analysis revision needs an author (`evidence_packets_analysis_guard`); no DELETE
`packet_findings` (`PacketFinding`)	A rule result inside a packet	`rule_id`, `title`, `direction` OVERCHARGE/UNDERBILLED, `amount_cents`, `calculation` (JSON steps), `confidence` decimal(4,3) (a fixed rule setting, not a measured probability), `needs_human_review`, `citations`, clause fields. Append-only

Table	Purpose	Notable fields and rules
`packet_sources` (`PacketSource`)	A document cited by a packet	`filename`, `doc_type`, `sha256`, `size_bytes`, `ordinal`. Append-only
`packet_timeline_events` (`PacketTimelineEvent`)	Ordered timeline	`occurred_at`, `kind` (APPOINTMENT, ARRIVAL, DEPARTURE, INVOICE, RATE_CONFIRMATION, NOTE), `label`, `source_id`. Append-only
`approvals` (`Approval`)	Every human decision on a packet	`action` EDIT/APPROVE/REJECT/SEND_READY, `reason` (required), `from_status`, `to_status`, `content_hash`, `actor_id`, `actor_role`, `packet_revision`, `created_at`. Append-only; `approvals_four_eyes_guard` and `approvals_four_eyes_ext_guard` refuse APPROVE or REJECT by an author

3.3 Imports and documents

Table	Purpose	Notable fields and rules
`import_batches` (`ImportBatch`)	A group of uploaded files	`source` UPLOAD/EMAIL (email ingest is designed in `docs/design/email-ingest.md`, not built), `created_by_id`. Append-only
`import_documents` (`ImportDocument`)	One uploaded file	`sha256` unique per batch, `detected_type` PDF/PNG/JPEG/CSV/TXT, `doc_type` with `doc_type_basis`, `status` RECEIVED/NEEDS_REVIEW/ACCEPTED/REJECTED/FAILED, `review_reasons`, `storage_key` (CHECK: starts with `t/<tenant_id>/`), provider and parser versions. Guarded updates, no DELETE
`extracted_fields` (`ExtractedField`)	One field pulled from a document	`key`, `kind`, `value`, `raw_value`, `corrected_value`, `confidence`, `needs_review`, `status` PROPOSED/CONFIRMED/CORRECTED/REJECTED, `origin` EXTRACTED/MANUAL, location (`page`, `line`, `start_col`, `end_col`, `source_row`, `excerpt`). Guarded updates, no DELETE
`import_review_decisions` (`ImportReviewDecision`)	Every review action	`action` CONFIRM/CORRECT/REJECT/ADD/SET_TYPE/ACCEPT/REJECT_DOC, `reason`, before and after value **hashes** (never the values), `actor_id`. Append-only; four-eyes guard on ACCEPT
`claim_documents` (`ClaimDocument`)	Link from an accepted document to a claim	A document links to at most one claim. Append-only

3.4 Audit, flags and evaluation

Table	Scope	Purpose	Notable fields and rules
`audit_events` (`AuditEvent`)	per chain (`t:<tenantId>` or `platform`)	Hash-chained audit log	`seq` contiguous per `chain_key`; `action` (one of the names in `packages/shared/src/audit-actions.ts`); `target_type`, `target_id`, whitelisted `metadata`, `prev_hash`, `hash`. Append-only; index on `(tenant_id, target_type, target_id)` serves the four-eyes readers
`feature_flags` (`FeatureFlag`)	global	Kill switches for intelligence features	7 keys in `packages/shared/src/feature-flags.ts`; `version` must rise by 1; no INSERT or DELETE
`eval_runs` (`EvalRun`)	global	One evaluation run on a synthetic set	pass^k counts, Wilson interval, `git_sha`, provider and parser versions. Append-only
`eval_case_results` (`EvalCaseResult`)	global	Per-case result of a run	`runs_passed`, `k`, `distinct_outputs`, `first_failure_code`. Append-only

3.5 Machine access

Table	Purpose	Notable fields and rules
`api_keys` (`ApiKey`)	Tenant API key	`key_id` (16 hex, unique, the public part), `secret_hash` (HMAC-SHA256 with a pepper held outside the DB), `scopes`, `expires_at`, `last_used_at`, `revoked_at`, `revoked_by_id`, `revoke_reason`, `created_by_id`. Guarded updates, no DELETE

3.6 Intelligence

Table	Scope	Purpose	Notable fields and rules
`charge_labels` (`ChargeLabel`)	tenant	Team label memory for charge-line wording	`normalized_text` unique per tenant, `category`, `description`, `version`; capped per tenant; label authors count for four-eyes
`fsc_tables` (`FscTable`)	tenant	Customer fuel-surcharge tables	`kind`, `unit`, `price_series`, `effective_offset_days`, `params` (JSONB), `archived_at`, `version`
`claim_fuel_inputs` (`ClaimFuelInput`)	tenant	Ship date and miles for a claim's fuel check	One per claim; `fsc_table_id` with `ON DELETE RESTRICT`
`price_imports` (`PriceImport`)	global	Provenance of a public price-series import	`series_key`, `source_url` (CHECK: two pinned FRED URLs), `sha256`, `row_count`. Append-only; `freight_app` has SELECT only
`price_observations` (`PriceObservation`)	global	Weekly or monthly price values	Key `(import_id, period_date)`, `value` decimal(10,3). Append-only

3.7 Ocean demurrage and detention

Table	Purpose	Notable fields and rules
`dd_tariffs` (`DdTariff`)	Carrier or terminal D&D tariff	`carrier`, `charge_type`, `direction`, `data` (JSON free time and tiers), `version`
`dd_calendars` (`DdCalendar`)	Terminal holiday calendar	`data` (JSON), `version`
`dd_invoices` (`DdInvoice`)	A D&D invoice under audit	`status` DRAFT or AUDITED (AUDITED rows are immutable, trigger `dd_invoices_guard`); `invoice_number` with `billing_party_name` unique per tenant; `invoice_date`; `total_due_cents`; `data` (JSON containers, events and `dueDate`, size-capped by CHECK); `tariff_id`; `calendar_id`; `claim_id` (unique); `audited_at`; `audit_snapshot`

4. Status machines

Column	States	Allowed moves (enforced by trigger)
`claims.status`	AWAITING_ANALYSIS, PENDING_REVIEW, APPROVED, REJECTED, SEND_READY	AWAITING_ANALYSIS to PENDING_REVIEW (analyze); PENDING_REVIEW to APPROVED or REJECTED; APPROVED to SEND_READY (terminal); an edit returns the claim to PENDING_REVIEW with a new packet revision (`claims_status_guard`)
`evidence_packets.status`	PENDING_REVIEW, APPROVED, REJECTED, SEND_READY, SUPERSEDED	Same transitions; an edit creates a new revision and supersedes the old one; no other column may change (`evidence_packets_update_guard`)
`import_documents.status`	RECEIVED, NEEDS_REVIEW, ACCEPTED, REJECTED, FAILED	`import_documents_update_guard`; ACCEPT needs a reviewer who did not upload or edit the document
`dd_invoices.status`	DRAFT, AUDITED	AUDITED rows are immutable (`dd_invoices_guard`)

5. What the schema does not hold

These absences matter for product claims and for the executive dashboard (../exec-dashboard-spec.md).

- **No outcome after send-ready.** Nothing records that a demand went out, that a carrier answered, what amount was credited or paid, or when. `SEND_READY` is the last state, so "recovered dollars" in the sense of money actually returned cannot be computed from this schema.
- **No dispute deadline.** No column holds a filing deadline or a contract dispute window. The ocean D&D engine knows the 46 CFR 541.8(a) minimum of 30 calendar days from the invoice date (`DD_DISPUTE_MIN_DAYS` in `packages/shared/src/ocean-dd/constants.ts`) and reads the invoice's `dueDate` from `dd_invoices.data`. Freight claims carry only `claims.invoice_date`.
- **No lane.** Claims and extracted fields have no origin or destination. The BOL `facility` field (`bol.facility` in `api/src/imports/parse/deterministic.ts`) is the closest location datum.
- **No team or business unit.** There is no Team model, so every tenant role sees every claim of its tenant (assumption A4 in `ARCHITECTURE.md`).
- **No billing or contract data.** No plan, subscription, invoice or contingency-fee table exists.

API surface (TypeScript API)

> **Status.** Pre-product, synthetic data only, never deployed; there is no public base URL. This inventory is generated from the `defineRoute(...)` calls in `api/src/**` on branch `feat/hardening` (`ed3d126`) by a script that reads `method`, `url`, `access` and `rateGroup` from source. The purpose column is ours. The Python service's API (`/v1/analyze` and friends) is a different API, covered in dev-portal-drift.md.

1. Summary

The TypeScript API exposes **97 routes** in 15 route files. One of them (`GET /api/v1/dev/outbox`) is registered only when the dev mail outbox is enabled, which production configuration refuses, so a production build serves 96. All application routes live under `/api/v1`; the two probes are `/healthz` and `/readyz`. There is no OpenAPI document for this API yet.

Access kind	Routes	Meaning
`public`	12	No session: auth bootstrap (csrf, login, MFA, forgot, reset, invites), probes, dev outbox
`cookie-session`	2	Refresh and logout, authenticated by the `fr_rt` cookie only
`authenticated`	2	Any signed-in user (`/me`, change password)
permission	81	A named permission from `packages/shared/src/rbac.ts`, checked on every request

Machine access: 9 routes accept a tenant API key (`Authorization: Bearer fr_live_...`), each only with the matching scope. `claims.read` opens `GET /claims`, `GET /claims/:id` and `GET /claims/:id/documents`; `exports.claims` opens `GET /exports/claims`; `imports.write` opens the five upload and batch routes. Every other route answers 403 to a key.

Rate groups (`api/src/http/context.ts`): `auth` and `forgot` are counted per client address; `upload`, `export`, `platform`, `intelligence` and `analysis` are counted per user; a global per-address limit (`RATE_LIMIT_GLOBAL_MAX`, default 600) applies to everything. The store is Redis.

2. Conventions every route shares

- **Order of checks.** Rate limit, then Origin and CSRF (unsafe methods, cookie callers only), then authentication, then permission, all in `onRequest` before the body is read; then Zod validation of body, query and params in `preHandler` (`api/src/http/route.ts`, `api/src/http/security.ts`).
- **Path parameters.** UUIDs in any letter case are lower-cased at the boundary (`canonicalizeParams`). A malformed or foreign id answers 404, never 403.
- **Errors.** One JSON shape from `api/src/http/errors.ts` with a fixed `code` and message; validation errors list field paths and Zod codes, never submitted values. Database guard violations map to fixed 4xx codes (`api/src/http/db-guard.ts`), for example `four_eyes_required` (403).
- **Writes.** Every mutating route appends an audit event in the same transaction. Decision routes require a `reason`. Editable configuration requires the current `version` and answers 409 when it is stale.
- **Body limits.** A global `BODY_LIMIT_BYTES` applies; allow-listed routes may raise it up to `ROUTE_BODY_LIMIT_MAX` (the D&D invoice routes). Uploads have their own size, idle and request timeouts.

3. Route inventory

`api/src/apikeys/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
POST	`/api/v1/api-keys`	`apikeys:manage`			Create a tenant API key (secret shown once)
GET	`/api/v1/api-keys`	`apikeys:manage`			List tenant API keys (no secrets)
POST	`/api/v1/api-keys/:id/revoke`	`apikeys:manage`			Revoke a key with a reason

`api/src/audit/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/audit/events`	`audit:read`			Page the tenant audit chain
GET	`/api/v1/audit/verify`	`audit:read`			Recompute the tenant chain; report first broken seq

`api/src/auth/dev-outbox.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/dev/outbox`	public			Dev/test mail outbox (registered only when the dev outbox is enabled)

`api/src/auth/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/auth/csrf`	public		auth	Issue CSRF cookie and token
POST	`/api/v1/auth/login`	public		auth	Password login (lockout, dummy verify)
POST	`/api/v1/auth/mfa/verify`	public		auth	Verify TOTP or recovery code
POST	`/api/v1/auth/mfa/enroll/start`	public		auth	Start TOTP enrolment
POST	`/api/v1/auth/mfa/enroll/verify`	public		auth	Finish TOTP enrolment
POST	`/api/v1/auth/refresh`	cookie-session		auth	Rotate refresh token; reuse revokes family
POST	`/api/v1/auth/logout`	cookie-session		auth	Revoke refresh family
POST	`/api/v1/auth/forgot`	public		auth	Request reset mail (same answer for unknown emails)
POST	`/api/v1/auth/reset`	public		auth	Reset password; revokes all sessions
POST	`/api/v1/auth/invites/inspect`	public		auth	Check an invite token
POST	`/api/v1/auth/invites/accept`	public		auth	Accept an invite
POST	`/api/v1/auth/change-password`	authenticated		auth	Change own password (own lockout)

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/me`	authenticated			Current user, role, tenant

`api/src/auth/users-routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/users`	`users:read`			List tenant users
POST	`/api/v1/users/invites`	`users:manage`			Invite a user (rank-based role assignment)
GET	`/api/v1/users/invites`	`users:read`			List invites
DELETE	`/api/v1/users/invites/:id`	`users:manage`			Revoke an invite
PATCH	`/api/v1/users/:id/role`	`users:manage`			Change a user role
POST	`/api/v1/users/:id/mfa/reset`	`users:manage`			Reset a user MFA enrolment
POST	`/api/v1/users/:id/disable`	`users:manage`			Disable a user
POST	`/api/v1/users/:id/enable`	`users:manage`			Re-enable a user

`api/src/claims/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/claims`	`claims:read`	`claims.read`		List, search, filter, sort claims
GET	`/api/v1/claims/:id`	`claims:read`	`claims.read`		Claim detail
GET	`/api/v1/claims/:id/packet`	`claims:read`			Current packet with findings, sources, timeline, integrity
POST	`/api/v1/claims/:id/assign`	`claims:assign`			Assign a claim
POST	`/api/v1/claims/:id/packet/revisions`	`packets:edit`			Edit: create a new packet revision
POST	`/api/v1/claims/:id/packet/approve`	`packets:approve`		analysis	Approve (reason, content hash, four-eyes)
POST	`/api/v1/claims/:id/packet/reject`	`packets:approve`		analysis	Reject (reason, four-eyes)
POST	`/api/v1/claims/:id/packet/send`	`demands:send`		analysis	Mark send-ready after hash match (sends nothing)
POST	`/api/v1/claims/analyze-awaiting`	`claims:analyze`		analysis	Bulk analyze AWAITING_ANALYSIS claims
POST	`/api/v1/claims/:id/analyze`	`claims:analyze`		analysis	Analyze one claim into packet revision 1
GET	`/api/v1/approvals`	`claims:read`			Approval queue

`api/src/exports/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/exports/claims`	`export:claims`	`exports.claims`	export	Stream claims as CSV/XLSX
GET	`/api/v1/exports/packets`	`export:packets`		export	Stream packet findings
GET	`/api/v1/exports/outcomes`	`export:outcomes`		export	Stream decision history (approvals rows)

`api/src/imports/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
POST	`/api/v1/imports`	`import:run`	`imports.write`		Create an import batch
GET	`/api/v1/imports`	`import:run`	`imports.write`		List batches
GET	`/api/v1/imports/:batchId`	`import:run`	`imports.write`		Batch detail
GET	`/api/v1/imports/:batchId/documents/:docId`	`import:run`	`imports.write`		Document detail with fields
GET	`/api/v1/imports/:batchId/documents/:docId/original`	`import:run`			Proxied download of the original
PATCH	`/api/v1/imports/:batchId/documents/:docId`	`import:review`			Set document type
POST	`/api/v1/imports/:batchId/documents/:docId/fields`	`import:review`			Add a field manually
POST	`/api/v1/imports/:batchId/documents/:docId/fields/:fieldId/resolve`	`import:review`			Confirm, correct or reject a field
POST	`/api/v1/imports/:batchId/documents/:docId/accept`	`import:review`			Accept a document (four-eyes)
POST	`/api/v1/imports/:batchId/documents/:docId/reject`	`import:review`			Reject a document
GET	`/api/v1/reviews`	`import:review`			Document review queue
POST	`/api/v1/imports/:batchId/commit`	`import:run`		upload	Commit accepted documents to claims
GET	`/api/v1/claims/:id/documents`	`claims:read`	`claims.read`	upload	Documents linked to a claim
POST	`/api/v1/imports/:batchId/documents`	`import:run`	`imports.write`	upload	Upload a file (sniffed, sandbox-parsed)

`api/src/intelligence/charge-routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/claims/:id/charge-lines`	`claims:read`		intelligence	Charge-line suggestions for a claim
GET	`/api/v1/charge-labels`	`claims:read`		intelligence	List team charge labels

Method	Path	Access	API-key scope	Rate group	Purpose
PUT	`/api/v1/charge-labels`	`charge_labels:manage`		intelligence	Set a team charge label
DELETE	`/api/v1/charge-labels/:id`	`charge_labels:manage`		intelligence	Delete a team charge label

`api/src/intelligence/fuel-routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/fuel-tables`	`claims:read`		intelligence	List fuel-surcharge tables
POST	`/api/v1/fuel-tables`	`settings:manage`		intelligence	Create a fuel table
PUT	`/api/v1/fuel-tables/:id`	`settings:manage`		intelligence	Update a fuel table
DELETE	`/api/v1/fuel-tables/:id`	`settings:manage`		intelligence	Archive a fuel table
GET	`/api/v1/claims/:id/fuel-check`	`claims:read`		intelligence	Fuel-surcharge check for a claim
PUT	`/api/v1/claims/:id/fuel-inputs`	`claims:analyze`		intelligence	Set ship date, miles, fuel table
DELETE	`/api/v1/claims/:id/fuel-inputs`	`claims:analyze`		intelligence	Remove fuel inputs

`api/src/intelligence/outlook-routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/outlook`	`claims:read`		intelligence	Cost outlook from public price series

`api/src/intelligence/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/features`	`claims:read`		intelligence	Tenant-visible feature flags
GET	`/api/v1/intelligence/worklist`	`claims:read`		intelligence	Prioritised worklist (priority-v1)
GET	`/api/v1/claims/:id/similar`	`claims:read`		intelligence	Similar past claims (similar-v1)
GET	`/api/v1/claims/:id/provenance`	`claims:read`		intelligence	Provenance summary

`api/src/ocean-dd/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/dd/tariffs`	`claims:read`		intelligence	List D&D tariffs
POST	`/api/v1/dd/tariffs`	`dd_tariffs:manage`		intelligence	Create a D&D tariff
PUT	`/api/v1/dd/tariffs/:id`	`dd_tariffs:manage`		intelligence	Update a D&D tariff
GET	`/api/v1/dd/calendars`	`claims:read`		intelligence	List terminal calendars

Method	Path	Access	API-key scope	Rate group	Purpose
POST	`/api/v1/dd/calendars`	`dd_tariffs:manage`		intelligence	Create a calendar
PUT	`/api/v1/dd/calendars/:id`	`dd_tariffs:manage`		intelligence	Update a calendar
GET	`/api/v1/dd/invoices`	`claims:read`		intelligence	List D&D invoices
POST	`/api/v1/dd/invoices`	`dd_invoices:manage`		intelligence	Create a D&D invoice
GET	`/api/v1/dd/invoices/:id`	`claims:read`		intelligence	D&D invoice detail with live preview
PUT	`/api/v1/dd/invoices/:id`	`dd_invoices:manage`		intelligence	Update a draft D&D invoice
DELETE	`/api/v1/dd/invoices/:id`	`dd_invoices:manage`		intelligence	Delete a draft D&D invoice
POST	`/api/v1/dd/invoices/:id/audit`	`claims:analyze`		analysis	Run the audit; creates OCEAN_DD claim and packet rev 1

`api/src/platform/dashboard-routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/api/v1/platform/pipeline`	`platform:health`		platform	Cross-tenant pipeline counts and durations
GET	`/api/v1/platform/telemetry`	`platform:health`		platform	Metrics registry snapshot
GET	`/api/v1/platform/logs`	`platform:logs`		platform	Derived log ring buffer
GET	`/api/v1/platform/flags`	`platform:flags`		platform	Feature flags
PUT	`/api/v1/platform/flags/:key`	`platform:flags`		platform	Change a flag (reason, version)
GET	`/api/v1/platform/eval/runs`	`platform:eval`		platform	Evaluation runs
GET	`/api/v1/platform/eval/runs/:id`	`platform:eval`		platform	Evaluation run detail
GET	`/api/v1/platform/audit/events`	`platform:audit`		platform	Platform audit chain
GET	`/api/v1/platform/audit/verify`	`platform:audit`		platform	Verify platform chain

`api/src/platform/routes.ts`

Method	Path	Access	API-key scope	Rate group	Purpose
GET	`/healthz`	public			Liveness
GET	`/readyz`	public			Readiness
GET	`/api/v1/platform/health`	`platform:health`			Platform health
GET	`/api/v1/platform/tenants`	`platform:tenants:list`			List tenants
GET	`/api/v1/platform/tenants/:tenantId/claims`	`platform:cross_tenant_read`			Audited read-only cross-tenant claim read

Developer portal drift

> **Status.** Pre-product, synthetic data only, never deployed. Compared on 2026-10-17: `developer-portal/openapi.json` and `developer-portal/docs/*.md` in the main checkout (the build worktree's copy is identical after JSON normalisation) against the TypeScript API on `feat/hardening` (`ed3d126`).

1. The short version

The developer portal and the TypeScript API describe two different products. The portal documents the **Python reference service** (`src/freight_recovery/api/main.py`): five operations, operator-issued `frk_` keys, a synchronous "upload three files, get a packet back" call, no rate limiting. Its OpenAPI file is generated from that FastAPI app, and CI keeps it honest against the Python code (`python developer-portal/build.py --check` in `.github/workflows/ci.yml`, job `developer-portal`). Nothing checks it against the TypeScript API, which is the product we would actually ship. The overlap is zero paths: no route in `openapi.json` exists in the TS API, and no TS route appears in the portal.

The portal is accurate about what it covers, and its "pre-product" warnings are correct. The problem is scope. A buyer or integrator who reads it will form a wrong picture of authentication, error shapes, rate limits and, most importantly, of what a machine client can do.

2. What the portal documents that the TS API lacks

Portal item (source)	Python behaviour	TS API today
`POST /v1/analyze` (`openapi.json`)	Multipart upload of up to 10 files; returns extracted values, findings, totals, Markdown packet and draft letter in one response	No equivalent. The TS flow is batch, upload, review, commit, then analyze, across several routes. `POST /claims/:id/analyze` takes no files, and **no API-key scope opens it**
`POST /v1/analyze/text`	Inline JSON documents	No equivalent; uploads are one `application/octet-stream` body per file
`GET /v1/analyses`, `GET /v1/analyses/{analysis_id}`	Paged list and fetch of stored analyses	Closest: `GET /api/v1/claims` and `GET /api/v1/claims/:id` (scope `claims.read`). The packet itself (`GET /claims/:id/packet`) is **not** open to keys
`GET /health` returning `{status, version}`	Anonymous	`GET /healthz` returns `{status}` only; the versioned route is `GET /api/v1/platform/health` and needs `platform:health`
`X-API-Key` header (`authentication.md`)	Accepted, as is `Authorization: Bearer`	Only `Authorization: Bearer fr_live_...` is recognised as a key
Key format `frk_<8 hex>_<43 chars>`	HMAC-SHA256 with a pepper	`fr_live_<keyId>_<secret>`, also HMAC with a pepper, but a different prefix and layout (`api/src/apikeys/key-material.ts`)
Keys issued by an operator CLI (`python -m freight_recovery.admin issue-key`)	No self-service	OWNER or ADMIN create, list and revoke keys in the app or through `/api/v1/api-keys`, with scopes and optional expiry
`401` for no key, `403` for any invalid key	Fixed messages	An invalid, revoked or expired key answers **401** with one identical body; `403` means "route not open to keys" or "scope missing" (`api/src/auth/api-key-auth.ts`)
Error body `{"detail": ...}` (`errors.md`)	FastAPI shape	`{"error": {"code", "message", "requestId", "details?"}}` with stable machine-readable codes (`sendError` in `api/src/app.ts`)

Portal item (source)	Python behaviour	TS API today
"No rate limiting, never returns 429" (`rate-limits.md`)	True for Python	A global per-address budget, per-group budgets, a per-key budget and an IP failure gate, all answering 429 with `Retry-After`
Limits: 10 files, 10 MiB per file, 25 MiB body, 30 s parse, 1024 MB worker	`FR_SANDBOX_*`	10 files per batch, 10 MiB per file (ceiling 25 MiB), 20 s parse timeout, 256 MB worker memory, 50 PDF pages, a 1 GiB storage quota per tenant (`api/src/config.ts`)
`perspective` = `shipper` or `carrier` per request	Request field	Stored on the claim as `SHIPPER` or `CARRIER`; no key-accessible route takes it as a parameter

3. What the TS API has that the portal does not mention

The portal covers none of the 97 TS routes, because they do not exist in the Python service. The groups an integrator or a buyer's diligence team would look for are below; the full list is in api-surface.md.

- **Import pipeline** (`/api/v1/imports/...` and `/reviews`, 14 routes): batches, per-file upload with type sniffing and a sandboxed parse, field-level review, four-eyes document acceptance, commit to claims. Five of these routes accept keys (`imports.write`).
- **Claims, packets and approvals** (11 routes): list and filter, packet revisions, approve, reject and send-ready with reasons, content-hash binding and four-eyes, single and bulk analyze.
- **Exports** (3 routes): streaming CSV and XLSX of claims, packet findings and decision history with formula neutralisation. `GET /exports/claims` accepts keys (`exports.claims`).
- **Intelligence** (16 routes): worklist, similar claims, provenance, charge-line suggestions and team labels, fuel tables and fuel check, cost outlook from public FRED series, tenant-visible feature flags.
- **Ocean demurrage and detention** (12 routes under `/api/v1/dd`): tariffs, calendars, invoices and an audit run against 46 CFR 541 and 545.5 that creates an `OCEAN_DD` claim.
- **Tenant administration and audit**: users, invites, roles, MFA reset, API keys, audit chain and verify.
- **Session auth for people** (13 routes): CSRF, login, TOTP MFA, refresh rotation, recovery, invites.

4. Recommendation

We recommend that the portal document the TypeScript API and that the Python service's pages move to an "internal reference implementation" note. Three facts drive this. The TS API is the stack the owner chose for everything customer-facing. A machine-readable contract already exists in `@fr/shared` (Zod schemas for every request and response), so generating a spec is cheap. And a buyer's technical due diligence will compare the portal with the running product; today that comparison fails on every route.

In this order:

1. **Stop the misleading signal (docs only).** Add a banner to `developer-portal/docs/index.md` saying the pages describe the Python reference service, not the product API, and link to an interim TS route list such as api-surface.md. No code change.
2. **Generate OpenAPI 3.1 from the TS routes.** Convert the Zod schemas attached to each `defineRoute` call (Zod 4 ships JSON Schema output) and emit `developer-portal/openapi.json` from a script, with access kind, permission and API-key scope as vendor extensions (`x-fr-permission`, `x-fr-api-key-scope`). Publish the 9 key-accessible routes first; the session routes only if we decide the web app's API is public.
3. **Move the drift check to the TS side.** Replace the Python `build.py --check` gate with a check in `web-ci.yml` that fails when the generated document differs from the committed one, the same contract the Python job enforces today.

4. **Rewrite the narrative pages** (`authentication.md`, `errors.md`, `rate-limits.md`, `quickstart.md`, `sdk.md`) for `fr_live_` keys, the `{"error": {...}}` shape, the 401 versus 403 rules, real 429 behaviour and the batch-upload flow.
5. **Settle the machine-client boundary before publishing** (first open question). With today's scopes an integration can push documents and read claim summaries, but it cannot trigger analysis, read a packet or record a decision. That can be the right line for a human-in-the-loop product. The portal should then state it plainly instead of implying the Python one-call flow.

These are changes to `developer-portal/` and a new script under `api/` or `tools/`; none is in this worktree's ownership, so we only recommend them here.

5. Open questions

- Should an API-key scope open `GET /claims/:id/packet` (read the packet) and `POST /claims/:id/analyze`? Today neither is possible by design. The approve, reject and send routes should stay human-only either way.
- Do we keep the Python FastAPI service runnable as a documented local demo, or reduce it to the parity oracle and retire its portal pages?
- Is the per-key rate limit a published, per-plan number? The portal should not state limits that pricing has not fixed.

Architecture decision records

These records reconstruct decisions already made in the code of the TypeScript monorepo (`ventures/freight-recovery-build`, branch `feat/hardening`, `ed3d126`) and its commit history. Each is marked **accepted (retroactive)**: nobody wrote it at the time; we wrote it on 2026-10-17 from the code, the commits and `ARCHITECTURE.md`. Where the repository does not record the alternatives that were weighed, the ADR says so rather than inventing a debate. The product is pre-product, synthetic data only and never deployed.

ADR	Decision	Decided
0001	TypeScript monorepo on Fastify 5, Prisma 7, PostgreSQL 16, React 19, with `@fr/shared`	2026-10-08
0002	Tenant isolation in three fail-closed layers anchored in PostgreSQL RLS	2026-10-08
0003	Python core kept as reference and exact-parity oracle	2026-10-09/10
0004	Hash-chained, append-only audit written in the business transaction	2026-10-08
0005	Four-eyes separation of duties on claim and document decisions	2026-10-09, extended 10-11 and 10-12
0006	Scoped tenant API keys bound to the creator's current role	2026-10-09
0007	Human approval gate bound to a content hash; nothing is sent	2026-10-08
0008	Deterministic engines; no trained model or LLM at runtime	2026-10-05 to 10-11
0009	Untrusted documents parsed in a bounded child process	2026-10-05 / 10-09
0010	One RBAC matrix; every route declares its access	2026-10-08
0011	Money as integer cents with exact decimal arithmetic	2026-10-05 / 10-08
0012	AWS target infrastructure in Terraform, validated, never applied	2026-10-08

New decisions get the next number and start as `proposed`.

ADR 0001: TypeScript platform on Fastify 5, Prisma 7 and PostgreSQL 16

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (scaffold commits `4798c70` to `b939225`, run `REQ-20261008-webapp-foundation`)
- **Reconstructed from:** `package.json` workspaces, `api/package.json`, `web/package.json`, `ARCHITECTURE.md` section "Web platform (TypeScript, steps 1-2)", the owner's stack decision in `BRIEF.md`

Context

By 2026-10-05 the repository held a working Python service (FastAPI, SQLAlchemy, Alembic) with API-key tenancy and sandboxed parsing, plus a small Vite pilot dashboard. A sellable product needed far more: people logging in with MFA, eight roles, a review and approval workflow, imports with field-level review, exports and an audit trail. Doing that in two languages with two copies of every role and schema would have doubled the places where a permission or a DTO could drift.

Decision

We build the customer-facing product as one npm-workspaces monorepo in TypeScript:

- `api/`: Node 22, Fastify 5.12, Prisma 7.10 on PostgreSQL 16, Zod 4 for validation, Pino for logs.
- `web/`: React 19.3 with Vite, React Router 7, TanStack Query 5 and Zustand 5.
- `packages/shared/` (`@fr/shared`): the one place for roles, the permission matrix, DTO schemas, canonical JSON and, later, the pure rule engines. Both `api/` and `web/` import it.

Prisma's removed `$use` middleware is replaced by a client extension (see ADR 0002). Node 22 was chosen over the brief's Node 20 because Node 20 reached end of life; CI pins 22.23.3 in `.nvmrc`.

Consequences

- One contract package means a role, permission or DTO is defined once. The web app's view of permissions cannot disagree with the API's, because it imports the same frozen object (it only hides controls; the API enforces).
- Prisma cannot express RLS, CHECKs or triggers, so the security-critical parts of the schema live in hand-written SQL migrations next to `schema.prisma`. Reviewers must read both.
- The Python service stays (ADR 0003), so the repository carries two runtimes and two CI workflows (`ci.yml`, `web-ci.yml`).
- Hiring and buyer diligence face a mainstream stack. Committed migration folders are immutable by project rule, so schema history is linear and reviewable.

Alternatives

Extending the Python FastAPI service into the full product was the obvious alternative. The commit history and `ARCHITECTURE.md` do not record a written comparison; the owner's brief for this package fixes the stack as decided, and we do not reopen it here.

ADR 0002: Tenant isolation in three fail-closed layers, anchored in PostgreSQL RLS

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (`35945d5` "Prisma schema + SQL migration (RLS fail-closed, least-privilege grants, append-only triggers)", `207479a`)
- **Reconstructed from:** `api/prisma/migrations/20261008000000_init/migration.sql`, `api/db/init/00-roles.sql`, `api/src/db/tenant.ts`, `api/src/auth/session.ts`, `ARCHITECTURE.md` "Tenant isolation (three layers, fail closed)", threat model T7 and section 4, `CLAUDE.md` hard rule 2

Context

The product holds several customers' freight invoices, rate confirmations and dispute packets in one database. A single missing `WHERE tenant_id = ...` in application code would leak one shipper's carrier rates to another, which would end the business. The Python service isolated tenants in its repository layer only and named RLS as "the next layer of defence".

Decision

Isolation is enforced three times, and each layer refuses access when it lacks information rather than allowing it:

1. **Database.** Every tenant table has `ENABLE` and `FORCE ROW LEVEL SECURITY` with `USING/WITH CHECK (tenant_id = fr_current_tenant())`. With no `app.tenant_id` set, queries return zero rows. The API connects as `freight_app` (`NOSUPERUSER NOBYPASSRLS NOINHERIT`, not the owner, DML grants only). Child tables reference parents by `[id, tenant_id]`.
2. **ORM.** A Prisma client extension (`api/src/db/tenant.ts`) allows only models listed in `TENANT_MODELS`, injects `tenantId` into every query and create, throws on a conflicting value and removes raw SQL. Every query runs in a transaction that sets `app.tenant_id` locally.
3. **Request.** User, role, tenant status and session liveness are reloaded from the database on every request, never trusted from token claims, headers or the body.

A new tenant table must ship with a policy, grants, a `TENANT_MODELS` entry and a cross-tenant test.

Consequences

- An application bug in one layer does not become a leak; it becomes a 404, an exception or zero rows.
- System-mode code paths (auth bootstrap on `memberships` and `invites`, the eval recorder, the price importer, the platform statistics function) are the residual risk the threat model names, and lint restricts where `db/system.ts` may be imported.
- Every request costs a few extra queries to reload context; nothing has been load-tested.
- Platform staff roles get no tenant database handle at all, which is also a data-minimisation property buyers ask about.

Alternatives

Schema-per-tenant or database-per-tenant isolation is not discussed in the repository. Application-only filtering (the Python approach) was the prior state and was explicitly treated as insufficient.

ADR 0003: The Python core stays as the reference implementation and parity oracle

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-09 to 2026-10-10 (`91743d4` "pure parsing core ... with Python parity", `d27c4a2` "port rules engine, letter and Markdown to @fr/shared/analysis with Python parity harness")
- **Reconstructed from:** `api/test/parity/extract.parity.test.ts`, `api/test/parity/rules.parity.test.ts`, `tools/parity/*.py`, `ARCHITECTURE.md` sections "Python parity: registered divergences" (D1-D8, RD1-RD13), `CLAUDE.md` (Python marked "Reference only", edits blocked by a hook), the owner's instruction in `BRIEF.md`

Context

The Python service in `src/freight_recovery/` already contained the domain logic that matters most: the detention clock, invoice checks, duplicate handling, the evidence packet and the draft letter, all in `Decimal`, with 29 tests and regression fixtures. The TS platform needed the same logic at runtime. Two independent implementations of money rules invite silent disagreement.

Decision

- The TS platform runs its own port of the logic in `@fr/shared` (`analysis/` and the parsing core in `api/src/imports/parse/`). The API never calls Python at runtime.
- The Python code is frozen as the reference. A repository hook blocks edits to `src/freight_recovery/**`.
- Parity harnesses run the Python reference through `tools/parity/*.py` and require **exact string equality** with the TS output on the fixtures, the Python test inputs and a seeded generated corpus. The only allowed differences are registered divergences, each with a reason (D1-D8 for extraction, RD1-RD13 for rules).
- In CI a missing Python interpreter fails the parity suite; locally it is skipped with a message.

Consequences

- A change to a money rule must either keep parity or add a registered divergence that a reviewer reads. This is a cheap, strong regression net for exactly the code a dispute letter quotes.
- Two runtimes stay in the repository, and the Python service still has its own API and developer portal, which has caused the drift described in `docs/architecture/dev-portal-drift.md`.
- Extensions written after the port (fuel check, ocean D&D, charge intelligence) have no Python counterpart; they rely on unit tests, acceptance suites and eval gates instead of parity.
- Rewriting the Python core is out of scope by owner decision.

Alternatives

Calling the Python service from Node over HTTP or a subprocess at runtime would have avoided the port but added a second deployable, a network hop and a second sandbox in the request path. The repository does not record a formal comparison.

ADR 0004: Hash-chained, append-only audit trail written in the business transaction

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (`35945d5`, `207479a`; fail-closed proof in `cf075e7`)
- **Reconstructed from:** `api/src/audit/audit.ts`, `api/src/audit/chain.ts`, `api/src/audit/routes.ts`, `packages/shared/src/audit-actions.ts`, migration triggers `audit_events_append_only` and `audit_events_no_truncate`, `ARCHITECTURE.md` "Audit hash chain", threat model T10

Context

Freight disputes are adversarial. When a shipper sends a demand built from our packet, the carrier may contest who approved it, what it said at approval time and whether anyone altered it afterwards. Buyers in this market also ask for an audit trail during security review.

Decision

- One chain per tenant (`chain_key = t:<tenantId>`) and one `platform` chain. `hash_n = SHA-256(hash_{n-1} + "\n" + canonicalJson(event_n))`, genesis is 64 zeros, and `seq` is contiguous under `pg_advisory_xact_lock(hashtextextended(chain_key, 0))`.
- Every mutating handler appends its event inside the same database transaction as the business change. If the append fails, the whole request fails with 500 and nothing changes.
- Triggers refuse UPDATE, DELETE and TRUNCATE on `audit_events` (and on `approvals`, `login_attempts` and the packet child tables) for every role, including the owner. Only the dev/test seed reset bypasses them.
- Metadata passes a whitelist, so document content and secrets never enter the chain.
- `GET /api/v1/audit/verify` recomputes the chain and reports the first broken `seq`.

Consequences

- Every write serialises on its tenant's chain lock. That is a throughput ceiling per tenant; it has not been measured.
- Approvals are tamper-evident from inside the application. Someone with a database superuser account could still rewrite a chain from genesis, because no chain head is anchored externally yet (threat model T10 residual; a WORM export is a known gap).
- The audit index on `(tenant_id, target_type, target_id)` (migration `20261017000000_audit_target_index`) lets four-eyes checks read authorship from the chain (ADR 0005).

ADR 0005: Four-eyes separation of duties on claim and document decisions

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-09 (`8eb7062`, run `REQ-20261009-four-eyes`, migration `20261011000000_four_eyes`); extended 2026-10-11 for D&D invoice editors (`10f8941`, `47a80bb`) and 2026-10-12 for label authors (`cd596c3`, `c3f131b`, migrations `20261018000000_hardening`, `20261019000000_hardening_fr2`)
- **Reconstructed from:** `packages/shared/src/four-eyes.ts`, `api/src/audit/label-authors.ts`, `api/src/audit/dd-editors.ts`, triggers `approvals_four_eyes_guard`, `approvals_four_eyes_ext_guard`, `import_review_decisions_four_eyes_guard`, `ARCHITECTURE.md` "Web platform: four-eyes" and "hardening", threat model section 8

Context

A demand letter asks a carrier for money. If the same person can build a packet and approve it, one mistaken or dishonest employee can put a wrong claim in front of a carrier with the company's name on it. Finance teams expect maker-checker controls on anything that becomes a payable or receivable.

Decision

- **Claims.** A user who created any revision of a claim's packet (current or superseded) cannot approve or reject the current revision. Since step 5 the analyzer counts as the author of revision 1. Since the hardening run, authors of team charge labels that influenced the analysis (`label_author`) and creators or editors of the ocean D&D invoice behind an `OCEAN_DD` claim are excluded too.
- **Imported documents.** The uploader and every user who recorded a field `CORRECT` or `ADD` cannot accept the document. An API-key upload counts as an upload by the key's creator.
- The service check is the primary control and runs under the claim row lock. Database triggers refuse the `approvals` or `ACCEPT` insert if the service check is bypassed. A refusal is one fixed 403 (`four_eyes_required`) for every caller and is audited (`authz.four_eyes_denied`). The UI hiding the button is cosmetic only.
- Policy defaults D1-D4 and H-D1, H-D2 are written down with their flip points (for example, send-ready stays unrestricted; when the 50-id label list may be incomplete, every tenant label author is excluded).

Consequences

- A tenant needs at least two active users with decision rights to move any claim to approved. A one-person customer cannot use the approval flow as designed; that is a product constraint, not a bug.
- Auto-accepted documents have no human acceptor (residual FE-S1, open question OQ-H4).
- Each new basis needs a service check, a trigger in a new migration, a basis text in `transparency-copy.ts` and an order entry in `FOUR_EYES_CLAIM_BASES`.

ADR 0006: Scoped tenant API keys bound to their creator's current role

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-09 (`a405edd` "tenant API keys (R80-R82, HMAC-only storage, scoped machine auth on nine routes, revocation without cache, attribution, abuse ...)"); fix rounds `1063b72`, `2ee385c`
- **Reconstructed from:** `api/src/auth/api-key-auth.ts`, `api/src/apikeys/*`, `packages/shared/src/api-key-dto.ts`, `ARCHITECTURE.md` "Request flow: API key vs cookie/JWT authentication", threat model T35

Context

Customers will want to push documents from a TMS or a mailbox integration and pull claim data into a finance system without a person's browser session. Machine credentials are a classic source of over-privilege: a key created by an admin keeps admin power after that admin leaves.

Decision

- Format `fr_live_<keyId>_<secret>`. Only `HMAC-SHA256(pepper, secret)` is stored; the pepper lives outside the database. The secret is shown once.
- A key is presented only as `Authorization: Bearer fr_live_...`. A key in the URL is refused with 400 and not logged. Key requests never use cookies or CSRF.
- At most three scopes exist (`claims.read`, `exports.claims`, `imports.write`), each mapped to one tenant permission. A route must opt in, and only 9 routes do.
- On every request: per-IP failure gate, constant-time compare (dummy digest for unknown ids), revocation and expiry read from the database without a cache, scope check, and a check that the creator's **current** role still holds the scope's permission. Disabled creator, downgraded role or suspended tenant: 401.
- Only OWNER and ADMIN manage keys, and a creator can grant only scopes they hold. Audit events carry actor role `API_KEY` and `viaApiKey`.

Consequences

- Offboarding a person disables their keys automatically while they are disabled. Re-enabling the person re-enables the keys; revoking keys on disable is open question OQ-H3.
- Machine clients cannot analyze, read packets or decide. That keeps humans in the loop but limits integrations (see `docs/architecture/dev-portal-drift.md`).
- The per-key database read on every request costs latency; it has not been measured.

ADR 0007: A human approval gate bound to a content hash; the product sends nothing

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (`207479a` "claims/packets/approvals", `24070f7` approvals UI); the Python service set the same principle on 2026-10-05 ("Human in the loop. Output is a draft packet; nothing is sent.")
- **Reconstructed from:** `api/src/claims/state-machine.ts`, `api/src/claims/content-hash.ts`, `api/src/claims/service.ts`, trigger `fr_packet_update_guard`, `ARCHITECTURE.md` "Approval state machine"

Context

Our output is an argument for money. A wrong number in a demand letter costs the customer credibility with a carrier they depend on, and an automated sender would turn a parsing error into a relationship problem. Separately, an approval is only meaningful if the thing approved cannot change afterwards.

Decision

- Packets move `PENDING_REVIEW` to `APPROVED` or `REJECTED`, and `APPROVED` to `SEND_READY` (terminal). Editing creates a new revision and supersedes the old one. Every action needs a reason and writes an append-only `approvals` row plus an audit event in one transaction, with the claim locked `FOR UPDATE`.
- Approve records the packet's canonical content hash (ordinals instead of UUIDs). Send recomputes it and requires it to equal both the approval's hash and the stored hash.
- "Send" only marks the demand send-ready. The product does not email, submit to portals or move money.
- The trigger `fr_packet_update_guard` enforces the same transitions in the database and makes every packet column except `status` immutable. Approving a packet with pending review findings requires an explicit `acknowledgePendingFindings`.

Consequences

- The approved artefact is provably the one marked send-ready; a later edit cannot ride on an old approval.
- Everything after send-ready (sending, carrier response, credit, payment) happens outside the system and is not recorded. The schema therefore cannot report recovered money (see `docs/exec-dashboard-spec.md`, gaps).
- Adding real sending later is a new trust boundary that needs its own design, consent and audit events.

ADR 0008: Deterministic, explainable engines; no trained model or LLM call at runtime

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-05 for the Python core ("Provider interface, deterministic default"); 2026-10-09 to 2026-10-11 for the TS intelligence features (`1ce0e52` priority-v1 and similar-v1, `959aa36` charge-knn-v1, `9625edd` evaluation tool, `53096a5` leave-one-carrier-out CI gate)
- **Reconstructed from:** `ARCHITECTURE.md` ("There is no trained model and no LLM call"), `api/src/intelligence/priority.ts`, `packages/shared/src/charge-intel/`, `api/src/eval/*`, `CLAUDE.md` hard rule 10, `technical/overview.md`

Context

Every amount in a packet must survive a carrier's dispute team asking "how did you get this number". A generative model that produces amounts cannot answer that question the same way twice. At the same time the project has no real documents to train or evaluate a model on, only synthetic fixtures.

Decision

- All money arithmetic runs in deterministic rule code (`@fr/shared/analysis`, `fuel/`, `ocean-dd/`). Each finding carries a rule id, an amount, a step-by-step calculation and a `needs_human_review` flag.
- "Intelligence" features are fixed, documented formulas: `priority-v1` (a weighted sum of stored amounts), `similar-v1`, and `charge-knn-v1`, which compares a charge name with labelled examples. Learned suggestions only ever add review items; they never change a rule's money output and never appear in the draft letter.
- Extraction sits behind a provider interface (`provider_name`, `provider_version` recorded per document). The only provider is deterministic. The Python `LLMExtractionProvider` is a placeholder that raises.
- Evaluation CLIs (`npm run eval`, `eval:charges`, `eval:outlook`) gate CI on synthetic sets and record pass^k statistics. They are regression checks, not accuracy measures, and the docs say so.

Consequences

- Outputs are reproducible and auditable, and parity with the Python reference is testable (ADR 0003).
- Real carrier PDFs with complex layouts, scans and photos will extract poorly until a layout-aware or OCR provider exists. That is the largest functional gap between the codebase and a paying customer.
- Any future model must enter through the provider interface, be recorded per document, and pass the same eval gates; it may not produce amounts directly.

ADR 0009: Untrusted documents are parsed in a bounded child process

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-05 in Python (`e2d760c` "Run parsing in a bounded worker process (timeout, memory cap, clean env)"); 2026-10-09 in TS (`8edf788` "parse sandbox: child-process executor, hardened worker, guard, hostile-worker tests")
- **Reconstructed from:** `api/src/imports/sandbox/*`, `api/src/imports/sniff.ts`, `api/src/config.ts` (parse limits), `ARCHITECTURE.md` "Import pipeline (data flow)", threat model section 6

Context

Customers upload carrier invoices, rate confirmations and bills of lading as PDF, image, CSV and text files from senders we do not control. PDF and image parsers have a long history of memory-exhaustion and code execution bugs. Parsing inside the API process would let one hostile file take down every tenant's requests.

Decision

- Uploads are size-checked, then type-sniffed from their bytes (`sniff.ts`); markup, binary content and type mismatches are refused before storage.
- Parsing runs in a separate child process per job (`sandbox/spawn.ts`, `worker-main.ts`) with a wall-clock timeout (20 s default, ceiling 60 s), a memory cap (256 MB default, ceiling 1024 MB), an output cap, page and pixel limits, a clean environment and a per-tenant concurrency gate (`KeyedGate`).
- The worker returns structured results over a checked protocol (`protocol.ts`, `guard.ts`); the parent validates them and records reason codes such as `parse_timeout`, `parse_memory` and `malformed_pdf`.

Consequences

- One bad file fails one document with a reason code instead of a whole request or process.
- This is process isolation, not an operating-system sandbox. seccomp, a read-only root filesystem and no-egress containment are deployment controls described for ECS but not verified (threat model).
- Throughput is bounded by `PARSE_MAX_CONCURRENCY` (default 2) per API instance; there is no queue, so large batches wait in the request. An async job queue is a known gap.

ADR 0010: One frozen RBAC matrix in the shared package; every route declares its access

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (`32e05b0` "@fr/shared - roles, permission matrix ...", `207479a` "RBAC enforcement")
- **Reconstructed from:** `packages/shared/src/rbac.ts`, `api/src/http/route.ts` (`defineRoute`, `enforceRouteAccess`), `api/src/http/security.ts`, threat model T6

Context

With six tenant roles, two platform roles and a growing route list, the usual failure is a new route that forgets its permission check, or a UI that shows a button the API would refuse. Both are cheap to prevent at the framework level and expensive to find in review.

Decision

- `rbac.ts` defines `TENANT_ROLES`, `PLATFORM_ROLES`, `PERMISSIONS` (30 today) and a frozen `PERMISSION_MATRIX`. The API calls `can(role, permission)` on every request; the web app imports the same data only to hide controls.
- Every route is registered through `defineRoute` with an `access` of `public`, `cookie-session`, `authenticated` or a named permission, optionally an `apiKeyScope`. The app refuses to boot if any route lacks a declaration.
- Authentication and permission checks run in the global `onRequest` hook before the body is parsed, so a forbidden caller gets 403 even for an invalid body. Denials are audited.
- Role assignment is rank-based: OWNER can never be assigned, and assigning ADMIN needs `admins:manage`.

Consequences

- Adding a feature means adding a permission to one file and declaring it on the route; tests enumerate routes times roles (T-RBAC-01..08).
- There is no Team model, so a role grants its permissions across all of the tenant's claims (assumption A4).
- Platform roles carry no tenant permission by construction, which keeps staff out of customer data.

ADR 0011: Money is stored as integer cents and computed in exact decimal arithmetic

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-05 in Python ("Money is stored as integer cents plus the exact packet JSON"; `Decimal` throughout `rules/`); carried into the TS schema on 2026-10-08 (`35945d5`) and the TS engines on 2026-10-10 (`d27c4a2`)
- **Reconstructed from:** `api/prisma/schema.prisma` (`*_cents Int` columns), `packages/shared/src/money.ts`, `packages/shared/src/decimal.ts`, Python `models.py`, `ARCHITECTURE.md` "Key decisions"

Context

Detention and accessorial disputes are often worth tens or hundreds of dollars per load. A floating-point rounding error of one cent, repeated across a batch, makes a letter look careless and invites the carrier to reject the whole claim.

Decision

- Every stored amount is an `Int` number of US cents (`amount_claimed_cents`, `recoverable_cents`, `pending_review_cents`, `amount_cents`, `total_due_cents`). `currency` defaults to `USD`.
- Rule arithmetic uses exact decimal code (`decimal.ts` in TS, `Decimal` in Python) and converts to cents once. Amounts cross the API as strings or integer cents, never as floats.
- Formatting goes through `formatUsdCents` in `@fr/shared`, so the web app and the letter render the same text.
- Analyses whose amounts fall outside the allowed range fail with the reason `amount_out_of_range` instead of being stored.

Consequences

- Sums and comparisons in SQL are exact, which the executive dashboard relies on.
- Multi-currency is not supported; a non-USD invoice would need a currency model and conversion rules.
- A 32-bit `Int` caps a single amount at about 21.4 million dollars, far above any single freight claim.

ADR 0012: AWS target infrastructure as Terraform, validated but never applied

- **Status:** accepted (retroactive)
- **Decided:** 2026-10-08 (`b939225` "Terraform skeleton (infra/)"), extended `fb1926d`, `c5d780b`, `b603b14`, `8ac4360`
- **Reconstructed from:** `infra/*.tf`, `infra/README.md`, `deploy/aws.md`, `ARCHITECTURE.md` "AWS mapping (web stack)", `CLAUDE.md` layout table

Context

Buyers ask where their data would live and how it would be protected before they look at features. The project has no customers and no budget for a running environment, but it needs a concrete, reviewable deployment design so that security claims in documents refer to something real.

Decision

- The web stack targets AWS: CloudFront with a response-headers policy generated from the same JSON the local nginx uses, S3 for the SPA with origin access control, an ALB reachable only from CloudFront with an origin-verify header, WAFv2, ECS Fargate in private subnets with VPC endpoints and no NAT, RDS PostgreSQL 16 with forced TLS and KMS, ElastiCache Redis 7 with TLS and at-rest encryption, an S3 documents bucket that denies puts without the customer-managed key, and seven Secrets Manager containers whose values are set out of band.
- Migrations run as a one-off ECS task before rollout. Secret values never enter Terraform state.
- Nothing is applied automatically. CI runs `terraform fmt -check` and `validate` only.

Consequences

- Every infrastructure statement in our documents is "designed in IaC, unverified in AWS". We must keep saying that until a real environment exists and has been tested.
- Running costs, failover behaviour and latency are unknown; none has been measured.
- A first deployment will surface issues that `validate` cannot catch (quotas, certificate validation, WAF rule tuning). The hardening run already added preconditions for some of these (H-2, H-3).

Innovation thesis

> **Status.** Pre-product, synthetic data only, never deployed, no paying customers. Nothing below has been measured on a real claim. Each innovation names a mechanism that exists in code today (branch `feat/hardening`, `ed3d126`), the research gap it answers (`research/02-industries.md` and `research/04-gaps.md`, market-researcher, 2026-10-17), and how we would measure it in a pilot. Where the code stops short of the gap, we say exactly where.
>
> **Revised 2026-10-17 after the owner's decision:** scoup.ai goes to market shipper- and importer-first. Carriers become a later segment, reached mainly through partners. The carrier-packet innovations moved to section 5.

1. Summary

The lead innovations serve the two first buyers: the mid-market shipper whose accounts-payable team receives carrier invoices with accessorials on top of the agreed rate, and the SME importer who receives ocean demurrage and detention (D&D) invoices. We rank them by the strength of the outside evidence behind each gap, and we say where that evidence is still thin.

#	Innovation	Gap answered	Evidence grade	Buyer it converts
I1	An accessorial and surcharge audit against the shipper's own contract terms, with a cited short-pay packet	Gap 5: auditing accessorials and fuel surcharge against the shipper's own rate confirmation and FSC table; mid-market shippers are too small for enterprise audit programmes	Outside evidence of workload (RXO survey of 750 shippers and carriers, a broker, not neutral); counter-evidence that shippers mostly trust carriers; no independent figure on overcharge rates	Controller or AP lead at a shipper with $5M to $50M freight spend
I 2	A regulation-coded ocean D&D invoice audit for SME importers	Gap 3: 46 CFR 541 checks and the 30-day window are sold to large importers only	Outside evidence: FMC, a law-firm analysis of the rule, a trade-association survey	Logistics or finance lead at an SME importer
I 3	One claims queue for the same shipper's truck and ocean charges	Gap 3, second half: no competitor ties the D&D window into a workflow that also handles truck charges for the same shipments	Competitive gap documented; no buyer quote yet	The same shipper or importer, as a reason to keep one vendor

Carrier-side ideas (the cited detention packet for the carrier who is owed money, machine-readable broker detention terms, a two-sided packet for brokers) keep their research backing (Gaps 1, 2 and 4) but move to section 5 as the later segment. Ideas without outside demand evidence stay hypotheses in section 6.

The market figures that bound this, as re-verified by market-researcher: the nine largest ocean carriers collected about $15.4 billion in D&D charges between April 2020 and March 2025 (an average of $3.08 billion a year, our arithmetic: $15.4B / 5); 240,535 US companies imported goods in 2024, 234,023 of them small and medium enterprises accounting for $884 billion of the $2,926 billion known import value (30.2%, market-researcher's arithmetic). Market-researcher's serviceable market (`research/05-market-size.md`, v4, 2026-10-17) is 32,765 accounts: 19,932 mid-market shippers, modelled from the Census SUSB 2022 count of manufacturers, wholesalers and retailers with $100M to $999.9M receipts at an ASSUMED 4.68% freight-to-sales ratio (average freight spend $9.53M), plus up to 12,833 mid-size importers (~2,000 TEU) capped by the FMC D&D pool. Priced per segment (finance `audit.md` C16), that is $945.4M a year: 19,932 x $27.6K = $550.1M for shippers plus 12,833 x $30.8K = $395.3M for importers, within a range of $0.55B to $0.95B. The freight ratio is an assumption, so the shipper count is a model, not a census.

Source: Federal Maritime Commission, Detention and Demurrage, https://www.fmc.gov/detention-and-demurrage/, accessed 2026-10-17 Source: U.S. Census Bureau, "A Profile of U.S. Importing and Exporting Companies, 2023-2024", release CB26-73, https://www.census.gov/foreign-trade/Press-Release/edb/edbrel2024.pdf, accessed 2026-10-17 Source: U.S. Census Bureau, Statistics of U.S. Businesses 2022, https://www2.census.gov/programs-surveys/susb/tables/2022/us_naicssector_large_rcptsize_2022.xlsx, accessed 2026-10-17 (extraction: `research/scripts/susb_shipper_counts.py`)

2. I1. Accessorial and surcharge audit against the shipper's own contract terms

The gap

Mid-market shippers buy truckload and LTL capacity and receive carrier invoices that add detention, lumper, liftgate, reweigh, redelivery and fuel charges to the agreed rate. Market-researcher found the incumbents aimed elsewhere: Loop at enterprise accounts, Intelligent Audit's smaller-shipper product parcel-first, CTSI-Global and nVision Global services-led (`research/04-gaps.md`, Gap 5).

The demand evidence is about **workload, not fraud**. RXO's survey of 750 shippers and carriers (25 November 2024) found that "61% of shippers agree that managing accessorial requests is one of the biggest time-wasters for their accounting team", that 65% of shippers see a moderate and 19% a significant budget impact from accessorials, that "71% are actively working on reducing accessorial spend", and that "80% of shippers would rather pay higher freight rates up front than deal with accessorials on the back end". RXO is a broker with no audit product to sell, but it is not neutral.

The same survey carries the counter-evidence we must keep in view: "70% of shippers agree that carriers are usually honest when requesting accessorials", only 8% saw outright dishonesty, and 67% think they pay the industry standard. So the pitch is "verify every accessorial and fuel line against your own terms with less effort", not "carriers overbill you". How much money that verification finds is ASSUMED (1% of freight spend in the business case); no independent source (CSCMP, Gartner, ARC, APQC or NASSTRAC) gives an overcharge rate, and the circulating 3 to 10% error rates are vendor figures.

An older benchmark supports the premise that shippers hold their own terms: Transplace's benchmarking of more than 150 shippers found the fuel surcharge to be the accessorial shippers most commonly publish (93% per search summaries; the article text was not readable, so we cite the headline only).

Source: RXO, "What Shippers & Carriers Really Think About Trucking Accessorials (Original Research)", https://rxo.com/resources/research/freight-accessorials/, accessed 2026-10-17 Source: SupplyChainBrain, "Fuel Surcharges Are Most Common Accessorial Published by Shippers: Benchmarking", https://supplychainbrain.com/articles/18205-fuel-surcharges-are-most-common-accessorial-published-by-shippers-benchmarking, accessed 2026-10-17 (2014; headline only)

Two pieces of the carrier-side research carry over with the direction flipped. Detention terms differ by broker and shipper (Gap 2), so a shipper checking a carrier's claim faces the same term-reading problem from the other side. And ATRI's finding that fleets are paid on "fewer than 50 percent" of detention invoices shows how many charges get contested. The business case's estimate of $16K a year in fuel-surcharge overbilling for a mid-size shipper is an internal assumption, not evidence.

The mechanism in this codebase

The shipper perspective already exists end to end. Every claim carries `perspective = 'SHIPPER'` or `'CARRIER'`, and the shipper rules look for money billed above what the contract supports.

1. **The contract terms are extracted values with locations.** The rate-confirmation vocabulary includes `linehaul_rate`, `fuel_surcharge`, `authorized_accessorials`, `detention_free_hours`, `detention_rate_per_hour` and `detention_max_hours` (`api/src/imports/parse/deterministic.ts`). Each lands in `extracted_fields` with page, line, column span and excerpt, and a second person must accept the document after any correction (ADR 0005).
2. **Six deterministic shipper checks.** `packages/shared/src/analysis/invoice-checks.ts` and `detention.ts` produce `INV-LINEHAUL-RATE` (linehaul above the rate confirmation), `INV-FUEL-SURCHARGE`, `INV-ACCESSORIAL-UNAUTH` (an accessorial line not on the confirmation's authorised list), `INV-DUPLICATE`, `INV-TOTAL-MISMATCH` and `DET-OVERBILLED` (detention billed beyond what the appointment, arrival and departure times support). Each finding carries its arithmetic step by step, in exact decimals (ADR 0011).
3. **The fuel surcharge is checked against the shipper's own fuel table.** Shippers store their fuel-surcharge tables (`fsc_tables`; step, row or miles-per-gallon forms). `packages/shared/src/fuel/check.ts` recomputes the expected surcharge from the table and the public weekly diesel price for the ship date (EIA series via FRED, with provenance in `price_imports`) and raises a review-only `FSC-INDEX-MISMATCH` when the billed amount differs.
4. **Unfamiliar charge names are classified, carefully.** Carriers name accessorials inconsistently. The `charge-knn-v1` classifier (`packages/shared/src/charge-intel/`) maps an unrecognised line to a category using labelled public tariff examples and the shipper team's own labels (`charge_labels`). Suggestions only add review items; they never change a rule's dollar output and never reach the letter.
5. **The output is a cited short-pay or dispute packet.** Each finding points back to the invoice line and the rate-confirmation field it rests on (`packet_findings.citations`, clause fields), the packet lists its source documents with hashes, and approval binds to a content hash (ADR 0007). The AP team can short-pay or dispute with the evidence attached.

Where the code stops short of the gap: extraction only reads labelled `Key: Value` lines, so real carrier invoices and rate confirmations in free layouts will extract little until a layout-aware or OCR provider exists (ADR 0008); shipper-carrier contracts that live outside the rate confirmation (master agreements, rate tables for many lanes) are not modelled; lumper and reweigh receipts have no rule that reads them; and invoices with no finding are not recorded, so overcharge as a share of audited spend cannot be reported yet (gap G9 in `docs/exec-dashboard-spec.md`).

Why it is hard to copy

The rules themselves are not secret. Three things together take time: every dollar traceable to a byte range in the customer's own documents, a fuel check pinned to a public price series with recorded provenance, and an approval record a carrier cannot argue was altered. The money rules sit inside an exact-parity test net against an independent Python implementation (ADR 0003), which is slow to build and easy to lose. Against enterprise audit firms the argument is price and fit: a self-serve, per-document product for a team too small to buy a managed audit programme. That argument is about our pricing and has not been tested.

Buyer it converts, and how we would measure it

The buyer is the controller or AP lead at a shipper spending $5M to $50M a year on freight. In a design-partner pilot, on the partner's own invoices:

- **AP time per invoice:** minutes the partner's accounting team spends verifying the accessorial and fuel lines of one invoice, before the pilot (timed sample) and with scoup.ai (from upload to a decided packet, `import_documents` and `approvals`). This is the measure the RXO evidence points to; we state no figure until it is measured.
- **Finding rate by check:** the share of audited invoices with at least one finding, per rule id, with the dollar value per rule. Needs a record of invoices audited with no finding (G9) to have a denominator.

- **Correction rate:** reviewer `CORRECT` actions per extracted field, per field key, from `import_review_decisions`.
- **Reviewer rejection rate:** the share of shipper packets the partner's own reviewers reject, from `approvals`, as an early signal of false findings.
- **Credited dollars:** what carriers actually credit after a short-pay or dispute. Needs outcome recording (G1) before it can be measured at all.

3. I2. A regulation-coded ocean D&D audit for SME importers

The gap

Under the FMC's final rule, an invoice that fails the content and timing requirements relieves the billed party's obligation to pay, and the billed party has at least 30 calendar days from issuance to request mitigation, refund or waiver. The D.C. Circuit vacated only 46 CFR 541.4 in September 2025; the rest of the rule stands. BlueCargo sells these checks to large importers ("1 in 5 Top US Shippers"); market-researcher found no competitor packaging them for the 234,023 SME importers in the Census profile, or tying the 30-day window into a workflow that also handles truck detention for the same shipments. Market-researcher rates this a partial gap that wins importer accounts rather than a business on its own.

Source: Thompson Hine, FMC final rule on D&D billing, https://www.thompsonhine.com/insights/fmc-issues-final-rule-on-billing-requirements-and-practices-for-detention-and-demurrage-invoices/, accessed 2026-10-17 Source: Thompson Hine, 26 September 2025, https://www.thompsonhine.com/insights/court-sets-aside-part-of-fmcs-demurrage-and-detention-billing-rule/, accessed 2026-10-17 Source: BlueCargo, home page, https://www.bluecargo.io/home-tests, accessed 2026-10-17 Source: U.S. Census Bureau, release CB26-73, https://www.census.gov/foreign-trade/Press-Release/edb/edbrel2024.pdf, accessed 2026-10-17

The mechanism in this codebase

1. **The regulation is code with a version.** `packages/shared/src/ocean-dd/regulation.ts` pins the clause text checked against the eCFR (`DD_REGULATION_VERSION = 'ecfr-2026-10-07'`), and `constants.ts` holds the 30-day limits of 541.7 and 541.8. 46 CFR 541.4 is deliberately not modelled because the court set it aside; the comment says so in both files.
2. **Seven named rules.** `DD_RULE_IDS`: `DD-TOTAL-MISMATCH`, `DD-PERIOD-ARITH`, `DD-TARIFF-OVERCHARGE`, `DD-545-RETURN-REFUSED`, `DD-545-AVAILABILITY`, `DD-541-7-LATE` and `DD-541-6-CONTENT` (`ocean-dd/audit.ts`), evaluated against the customer's tariff (`dd_tariffs`, with public presets and provenance in `presets.ts`) and a terminal calendar (`dd_calendars`).
3. **Same workflow as truck claims.** The audit run (`api/src/ocean-dd/audit-run.ts`) freezes the invoice (`AUDITED` is immutable in the database), creates an `OCEAN_DD` claim with packet revision 1, citations, CFR clauses, a timeline and a draft letter that reminds the sender of the 541.8(a) window. The claim then goes through the same approval gate and four-eyes rules, with invoice editors excluded from approving.
4. **Low-effort intake.** A CSV import (`ocean-dd/csv.ts`) brings in up to 50 invoices at a time, so a small importer does not need an integration to start.

Where the code stops short: there is no deadline alert or worklist ordering by days left (the executive dashboard spec proposes metric D1), no feed from carriers or terminals, and container events must be typed or imported by hand.

Why it is hard to copy

It is not, on its own: the rule is public law and BlueCargo already checks it for enterprises. The defensible part is narrower: one claims queue, one approval record and one audit chain covering both a shipper's ocean D&D invoices and the truck detention on the same moves, at a price an SME can carry. That argument depends on

pricing and on I1 and I3 holding up, not on the D&D code.

Who receives the dispute

Market-researcher found that an SME importer's first stop for a D&D dispute is often its forwarder or customs broker, and that a forwarder acting as an NVOCC is itself the billing party: CastleGate Forwarding takes D&D disputes "by entering a ticket through our website in Partner Home". The packet therefore has to work for a forwarder's dispute desk as well as an ocean carrier's. The code already treats the billing party as data (`dd_invoices.billing_party_name`, copied into `claims.carrier_name`), so a forwarder invoice audits the same way; what does not exist is any routing of the packet into a forwarder's ticket system. Forwarders also hold the importer relationship and the documents, which makes them a possible channel as much as a counterparty.

Source: CastleGate Forwarding, Regulatory, https://castlegateforwarding.com/resources/regulatory, accessed 2026-10-17

Buyer it converts, and how we would measure it

The buyer is an SME importer's logistics or finance lead who receives D&D invoices and has no specialist tool. Pilot measures, on the partner's own invoices:

- **Finding rate:** the share of audited invoices with at least one `DD-541-6-CONTENT` or `DD-541-7-LATE` finding, and the dollar value attached, reported per carrier.
- **Within-window rate:** the share of disputes made ready before day 30 of the invoice, from `claims.invoice_date` and the `SEND_READY` approval time.
- **Reviewer rejection rate:** the share of `OCEAN_DD` packets that the partner's own reviewers reject, from `approvals`, as an early signal of false findings.
- **Waived or refunded dollars:** needs outcome tracking (G1).

4. I3. One claims queue for a shipper's truck and ocean charges

The gap

Market-researcher's Gap 3 has a second half: no profiled competitor ties the 30-day D&D window into a claims workflow that also handles truck charges for the same shipments. BlueCargo serves D&D for large importers; the freight audit firms treat D&D as one more audit line or not at all. An SME importer that is also a domestic shipper today needs one tool, or one service firm, per charge type. We have no buyer quote asking for the combination, so this is a documented competitive gap with demand still to be confirmed in interviews.

Source: BlueCargo, home page, https://www.bluecargo.io/home-tests, accessed 2026-10-17

The mechanism in this codebase

One `claims` table holds both kinds (`claims.kind` = `FREIGHT` or `OCEAN_DD`, fixed by the trigger `claims_kind_guard`). Both kinds share the same packet tables, the same approval state machine and content-hash binding (ADR 0007), the same four-eyes rules (ADR 0005), the same per-tenant audit chain (ADR 0004), the same streaming exports (`api/src/exports/`) and the same worklist. The executive dashboard spec (`docs/exec-dashboard-spec.md`) reports both kinds side by side for a shipper executive, including the D&D mitigation window (metric D1) next to freight invoice age (D2).

Where the code stops short: claims of the two kinds are not linked to a common shipment or bill of lading, so "the same shipment" is a human association today; there is no shipment key in the schema.

Why it is hard to copy

A specialist would have to add a second charge domain with its own rules and evidence; an audit firm would have to rebuild its D&D line into a regulation-coded check. Neither is impossible. The advantage is that both already share one approval record and one audit chain here, which matters to the controller who must answer for every short-payment. It is a retention argument more than a moat.

Buyer it converts, and how we would measure it

The buyer is the shipper or importer who has both kinds of charges. Pilot measures:

- **Two-kind adoption:** the share of pilot tenants that create claims of both kinds within 90 days.
- **Queue time by kind:** time from packet creation to first decision, per kind (dashboard metric T2 split by `kind`), to see whether one queue slows either charge type.
- **Vendor consolidation:** whether the partner drops a separate D&D or audit service during the pilot, recorded in interviews, not inferred.

5. Later segment: carriers and brokers

Carriers become a later segment, reached mainly through partners (owner decision, 2026-10-17). The research behind these ideas is the strongest we have on the carrier side, so we keep it intact rather than discard it. The mechanisms overlap almost completely with I1: the carrier perspective uses the same extraction, citation and approval machinery with the carrier rules (`DET-UNBILLED`, `DET-UNDERBILLED`) instead of the shipper rules. A partner channel (for example an ELD or factoring company) could bring these buyers without a separate sales motion; that channel is a business question, not an architecture one.

5.1 A source-cited detention packet for the carrier who is owed money

The gap

Carrier-side tools (Hey Bubba, FleetOptos, Detention Source) record geofence arrival and departure and stop at the timestamp; freight audit and payment firms do not sell to carriers. The demand evidence is about proof. ATRI reports that 94.5 percent of fleets charge detention but are paid on "fewer than 50 percent of those invoices", and drivers on a public forum describe writing arrival and departure times on the freight bill by hand so a claim survives.

Source: American Transportation Research Institute, release of 10 September 2024, https://truckingresearch.org/2024/09/new-research-documents-substantial-financial-and-safety-impacts-from-truck-driver-detention/, accessed 2026-10-17 Source: TheTruckersReport forum, "What's the #1 reason your detention claims get denied", https://www.thetruckersreport.com/truckingindustryforum/threads/whats-the-1-reason-your-detention-claims-get-denied-poll.2529156/page-3, accessed 2026-10-17

The mechanism in this codebase

The packet is a chain of pointers from every dollar back to a byte range in a document the customer uploaded, sealed so that what a person approved is provably what goes out.

1. **Every extracted value keeps its location.** `extracted_fields` stores `page`, `line`, `start_col`, `end_col`, `source_row` and an `excerpt` next to the value (`api/prisma/schema.prisma`; parsers in `api/src/imports/parse/`). The original file is stored with its `sha256`.
2. **People correct values without erasing history.** Confirm, correct and add actions write `import_review_decisions` with before and after hashes; a second person must accept the document (`api/src/imports/review-service.ts`, ADR 0005).

3. **Rules show their arithmetic.** `packages/shared/src/analysis/detention.ts` and `invoice-checks.ts` produce findings with a rule id, a step-by-step `calculation` and a trace of the fields they used. The carrier perspective has its own rules (`DET-UNBILLED`, `DET-UNDERBILLED`), so detention earned but never billed is a finding, not a footnote.
4. **The packet stores the pointers.** `api/src/claims/analysis.ts` turns each trace into `packet_findings.citations` and governing-clause fields (`clause_source_id`, `clause_excerpt`, `clause_locator`), lists the documents in `packet_sources` with their hashes, and orders appointment, arrival and departure in `packet_timeline_events`.
5. **Approval binds to content.** Approve records a canonical content hash; send-ready recomputes it and refuses on any difference (`api/src/claims/content-hash.ts`, ADR 0007).

Where the code stops short of the gap: there is no geofence or ELD log intake, a lumper receipt is only an `OTHER` document with no rule that reads it, and extraction reads `Key: Value` text, so a real carrier PDF with a free-form layout will extract little until a layout-aware or OCR provider exists (ADR 0008). The two-sided version for brokers (Gap 4) needs a shareable, read-only packet view, which does not exist.

Why it is hard to copy

Nothing here is secret, and a funded team could rebuild it. The barrier is that the packet needs four things that today live in different companies' products: document extraction with locations, contract-term rules, a human review workflow with separation of duties, and a tamper-evident approval record. Tracking vendors would have to add document intake and contract reading; audit firms would have to sell to the party they audit against. The research names exactly these two stopping points as the reason the gap is open. The part a copier cannot take quickly is the exact-parity test net around the money rules (ADR 0003): every rule change is checked string-for-string against an independent implementation, which is slow to build and quick to lose.

Buyer it converts, and how we would measure it

The buyer is the owner or office manager of a 5 to 100 truck carrier who has ELD data and no one to turn it into a claim. In a design-partner pilot we would measure, on the partner's own closed loads:

- **Citation coverage:** the share of packet findings whose every input field resolves to a stored source span. Computed from `packet_findings.citations` and `extracted_fields`; no baseline exists, so the pilot sets it.
- **Correction rate:** reviewer `CORRECT` actions per extracted field, by field key, from `import_review_decisions`.
- **Time to send-ready:** hours from the first upload to the `SEND_READY` approval, from `import_documents` and `approvals`.
- **Paid share of detention claims:** the outcome that matters, compared with the partner's own paid share for the same lanes before the pilot. This needs outcome recording (gap G1 in `docs/exec-dashboard-spec.md`) before it can be measured at all.

5.2 Broker detention terms as reviewed, cited data

The gap

Detention terms are not standard. Free time, hourly rate, cap, notice rules and disqualifying conditions differ by broker and shipper and live in rate confirmations and broker policies. A carrier quoted on a public forum was told by a broker that there is "no standard", then received a written refusal citing "company policy". The one carrier-side competitor that mentions rules, FleetOptos, describes "contract-specific billing rules (free time allowances, hourly rates)", which reads as hand configuration.

Source: TheTruckersReport forum, "Broker doesn't want to pay / one of those days", page 2, https://www.thetruckersreport.com/truckingindustryforum/threads/broker-doesn%E2%80%99t-want-to-pay-one-of-those-days.2519612/page-2, accessed 2026-10-17 Source: Geotab Marketplace, FleetOptos listing, https://marketplace.geotab.com/solutions/fleetoptos/, accessed 2026-10-17

The mechanism in this codebase

We treat a contract term as an extracted, reviewable value with a source location, not as a setting someone types into an admin screen.

1. **Terms are extracted fields.** The rate-confirmation vocabulary includes `detention_free_hours`, `detention_rate_per_hour`, `detention_max_hours`, `authorized_accessorials`, `linehaul_rate` and `fuel_surcharge` (`api/src/imports/parse/deterministic.ts`). Each lands in `extracted_fields` with its span and goes through the same confirm, correct and four-eyes accept flow as section 5.1 and I1.
2. **The calculation reads the confirmed terms.** `packages/shared/src/analysis/detention.ts` starts the clock at the later of appointment and arrival, applies the extracted free time and cap, and writes every step into the finding. A second, tariff-shaped calculator (`computeTariffDetention` in `analysis/tariff-detention.ts`) handles tiered terms; it is pure and tested but **not yet wired** into analysis.
3. **Unfamiliar charge wording is learned per team, safely.** When an invoice line does not match the keyword rules, the `charge-knn-v1` classifier (`packages/shared/src/charge-intel/`) compares it with labelled examples and the team's own labels (`charge_labels`). These suggestions only ever add review items; they never change a rule's dollar output and never reach the letter, and their authors are excluded from approving the claim (`label_author`, ADR 0005).

Where the code stops short of the gap: terms are extracted per load, not kept as a per-broker policy that the next load can reuse; disqualifying conditions and notice requirements (for example "no detention if the driver left before loading") are not modelled; and extraction only recognises labelled `Key: Value` lines today.

Why it is hard to copy

The data shape is easy to copy. What takes time is coverage: the many ways brokers phrase free time, caps and exclusions, with each phrasing tied to a confirmed value. That library only grows from real rate confirmations that reviewers have corrected, and the review trail (`import_review_decisions`) is what would let us build it without storing anyone's document text in a shared corpus. We have none of that data yet, so today this is a design advantage, not an asset.

Buyer it converts, and how we would measure it

The buyer is the same carrier, specifically the person who prepares claims and today re-reads every rate confirmation. Pilot measures, on the partner's own documents:

- **Term extraction rate:** the share of rate confirmations where all three detention terms are extracted, and the share of those a reviewer confirms unchanged rather than corrects, per term.
- **Setup time per new broker:** minutes from a first rate confirmation to a confirmed term set, against the partner's current process. We make no claim about this number until it is measured.
- **Disputed-term rate:** the share of claims where the payer contests the term itself rather than the times, recorded once outcome tracking (G1) exists.

5.3 One packet the broker can use on both sides of the load

Market-researcher's Gap 4: two-thirds of brokers in a DAT survey paid detention only when they could collect it from the shipper, so a packet that travels carrier to broker to shipper has a use. The same survey found only 20% of 50 brokers ranked detention a top-five problem, so demand is mixed. The code has no shareable,

read-only packet view across tenants; building one would cross the tenant boundary (ADR 0002) and needs its own design.

Source: DAT Solutions, 14 July 2016, https://www.dat.com/company/news-events/news-releases/63-percent-of-drivers-are-detained-for-more-than-3-hours-per-stop-dat-solutions, accessed 2026-10-17

6. Hypotheses without outside demand evidence

Market-researcher found no independent source asking for the following. We keep them out of the pitch until buyer interviews support them.

- **Four-eyes approval on outgoing disputes** (ADR 0005). It is built and enforced in the database. Whether a small carrier values a second-person gate, or sees it as friction for a one-person office, is unknown.
- **Cost outlook** (`packages/shared/src/outlook/`). A forecast of fuel and freight cost from public FRED series; no buyer has asked for it. (The fuel-surcharge *check* is part of I1; the size of fuel-surcharge errors is still sourced only from vendor blogs.)
- **Priority scoring** (`priority-v1`). A fixed, documented formula; no buyer has asked for it.
- **A dispute-outcome data moat.** It does not exist: the schema records nothing after send-ready (ADR 0007), and there is no claim volume. Building G1 is the precondition for this hypothesis to become testable.

7. What we will not claim

We do not claim accuracy, recovery rates, time savings or security in production. The eval gates run on synthetic fixtures and are regression checks, not accuracy measures. Every number in this thesis is either a sourced market figure or a metric we would measure in a pilot.

Executive dashboard: specification input

> **Status.** Proposed, not built. Pre-product, synthetic data only, never deployed. This document is input for `fr-planner`, who turns it into `spec.md` and `test-spec.md`. Every metric below names the tables and columns it reads in `api/prisma/schema.prisma` on `feat/hardening` (`ed3d126`). Anything the schema cannot answer is listed as a **gap** in section 6, not approximated silently.

1. Why this page exists

The people who sign for scoup.ai are a CFO, a controller or a head of logistics at a mid-market shipper or an SME importer (owner decision of 2026-10-17: shippers and importers first, carriers later through partners). They will not open claim sheets. They want one page that answers four questions each week: how much money have we found in our carriers' and terminals' invoices, where is it stuck, what is about to expire, and which carriers and charge types keep overbilling us. Today the product has a worklist for analysts (`/intelligence`) and a platform dashboard for our own staff (`/dev`), but nothing for a customer's executives.

The page has one hard rule that comes from the project's honesty rule: **it shows money identified and money approved for a demand, never money recovered.** The schema stops at `SEND_READY` (ADR 0007), so nothing in the database knows whether a carrier paid. Every dollar figure carries the label "identified" or "ready to demand", and the page footer states in plain words that recovery outcomes are not tracked yet.

2. Audience and access

Decision	Proposal	Reason
Permission	New permission `insights:read` in `packages/shared/src/rbac.ts`	Every route needs a declared permission (ADR 0010); a named permission lets a tenant give a CFO the page without approval rights
Roles that hold it	OWNER, ADMIN, MANAGER, VIEWER	VIEWER is the natural seat for an executive who must see results but must not approve packets (four-eyes, ADR 0005). REVIEWER and ANALYST already see every claim through `claims:read`; whether they also get the page is open question Q1
Platform roles	None	Platform roles hold no tenant data permission; this stays true
API keys	No scope in the first version	No `apiKeyScope` on the new routes; exporting the numbers to a BI tool is a later decision (Q2)
Tenant scope	Own tenant only, through `ctx.db` (tenant transaction, RLS)	Same three layers as every tenant read (ADR 0002)
MFA	Unchanged	MFA stays mandatory for OWNER and ADMIN only; VIEWER and MANAGER follow existing policy

Aggregates expose nothing a VIEWER cannot already read through `GET /api/v1/claims`, so the new permission adds no new data exposure for the four roles. It does add a new surface for rate limiting and query cost, handled in section 5.

3. Definitions

Term	Definition (exact)
Claim	A row in `claims` for the caller's tenant

Term	Definition (exact)
Kind	`claims.kind`: `FREIGHT` (truck detention, accessorial, fuel) or `OCEAN_DD` (ocean demurrage and detention)
Current packet	For a claim, the `evidence_packets` row with the highest `revision` (status is never `SUPERSEDED`)
Identified recoverable	`claims.recoverable_cents`: the sum of confirmed findings on the current packet. Equal to the current packet's `recoverable_cents`, because an edit copies amounts unchanged (`api/src/claims/service.ts`)
Pending review	`claims.pending_review_cents`: findings that need a human decision; never counted as identified
Ready to demand	Identified recoverable of claims with `status = 'SEND_READY'`
Approved, not yet send-ready	Identified recoverable of claims with `status = 'APPROVED'`
Open claim	`status IN ('AWAITING_ANALYSIS','PENDING_REVIEW','APPROVED')`
Closed in product	`status IN ('SEND_READY','REJECTED')` (closed in scoup.ai, not closed with the carrier)
Period	A half-open interval `[from, to)` on a named timestamp, in UTC, at most 366 days long
Carrier key	`lower(trim(claims.carrier_name))`. For `OCEAN_DD` claims this column holds the D&D invoice's billing party (`api/src/ocean-dd/audit-run.ts` copies `billingPartyName` into `carrierName`), so one key serves both kinds. For an importer the billing party is often a forwarder acting as NVOCC (`research/01-competitors.md`), so the UI calls this column "billing party", not "carrier". The display name is the most frequent original spelling in the group

4. Metric catalogue

All metrics are computed per tenant, inside one read-only tenant transaction, and filtered by an optional `kind` (`FREIGHT`, `OCEAN_DD` or both) and an optional `perspective` (`SHIPPER`, `CARRIER` or both). The page opens with `perspective=SHIPPER`, because the first customers are shippers and importers; every `OCEAN_DD` claim is created with `perspective = 'SHIPPER'` (`api/src/ocean-dd/audit-run.ts`), so importers see their D&D claims under the default. Money is summed as integer cents in SQL and returned as integer cents (ADR 0011); the web app formats with `formatUsdCents`.

4.1 Money

ID	Metric	Definition	Source
M1	Identified recoverable, open	Sum of `recoverable_cents` over open claims	`claims.recoverable_cents`, `claims.status`
M2	Pending review, open	Sum of `pending_review_cents` over open claims	`claims.pending_review_cents`, `claims.status`
M3	Approved, not yet send-ready	Sum of `recoverable_cents` where `status = 'APPROVED'`	`claims`
M4	Ready to demand (period)	Sum of `recoverable_cents` of claims whose `SEND_READY` approval falls in the period	`approvals.action = 'SEND_READY'`, `approvals.created_at`, joined to `claims.recoverable_cents`

ID	Metric	Definition	Source
M5	Identified in period	Sum of `recoverable_cents` of claims whose first analysis packet was created in the period	`evidence_packets.revision = 1`, `evidence_packets.created_at`, `claims.recoverable_cents`
M6	Rejected by our own reviewers (period)	Count and sum of `recoverable_cents` of claims whose latest decision in the period is `REJECT`	`approvals.action = 'REJECT'`, `approvals.created_at`
M7	D&D share disputed	For audited D&D invoices, sum of the linked claims' `recoverable_cents` divided by sum of `total_due_cents`, shown as a percentage with both totals; omitted when the denominator is 0 or any `total_due_cents` is null in the set (the count of excluded invoices is shown)	`dd_invoices.status = 'AUDITED'`, `dd_invoices.total_due_cents`, `dd_invoices.claim_id`, `claims.recoverable_cents`

4.2 Pipeline and stages

ID	Metric	Definition	Source
S1	Claims by stage	Count and `recoverable_cents` sum per `status` (all five states)	`claims.status`
S2	Ageing in stage	For each open status, counts in buckets 0-7, 8-30, 31-60 and over 60 days since the claim entered that status. Entry time is the latest `approvals.created_at` for the claim whose `to_status` equals the current status; for `AWAITING_ANALYSIS` it is `claims.created_at`; for `PENDING_REVIEW` without an approval row it is the current packet's `created_at`	`approvals.to_status`, `approvals.created_at`, `claims.created_at`, `evidence_packets.created_at`
S3	Document intake	Count of `import_documents` per `status` created in the period, and the count still `NEEDS_REVIEW` now	`import_documents.status`, `import_documents.created_at`
S4	Analysis failures (period)	Count of audit events `claim.analysis_failed` in the period, grouped by `metadata.reasonCode`	`audit_events.action`, `audit_events.created_at`, `audit_events.metadata`

4.3 Deadline exposure

ID	Metric	Definition	Source
D1	D&D mitigation window	For open `OCEAN_DD` claims with a non-null `claims.invoice_date`: days remaining until `invoice_date + 30 days` (`DD_DISPUTE_MIN_DAYS`, the 46 CFR 541.8(a) minimum). Buckets: already past, 0-7, 8-14, over 14 days. Count and `recoverable_cents` per bucket; claims with a null date counted separately as "date missing"	`claims.kind`, `claims.invoice_date`, `claims.status`, `packages/shared/src/ocean-dd/constants.ts`
D2	Freight invoice age	For open `FREIGHT` claims: days since `claims.invoice_date`, buckets 0-30, 31-60, 61-90 and over 90. Labelled "invoice age", **not** deadline, because the schema holds no contract dispute window (gap G2)	`claims.kind`, `claims.invoice_date`, `claims.status`

D1 uses the regulatory minimum only. A carrier's own tariff may allow more time; the page says "regulatory minimum" next to the number so nobody reads it as the actual deadline.

S4 reads `audit_events` the way `api/src/audit/analysis-failures.ts` already does (tenant transaction, `metadata ->> 'reasonCode'`), because `AuditEvent` is not a `TENANT_MODELS` entry.

4.4 Leakage

"Leakage" means money billed that our rules say should not have been, or money earned that was not billed, depending on the claim's `perspective`. For the default shipper view it means overcharges: linehaul above the rate confirmation, fuel surcharge above the shipper's own fuel table, accessorials the rate confirmation did not authorise, duplicate lines, totals above the sum of lines, detention billed beyond the supported time, and D&D charges that fail the tariff or 46 CFR 541 and 545.5. It is always "identified", never "confirmed by the carrier".

ID	Metric	Definition	Source
L1	Leakage by billing party (carrier, terminal or forwarder)	Top N carrier keys (default 10, max 50) by `recoverable_cents + pending_review_cents` over claims created in the period, with claim count, identified sum, pending sum and the share of the tenant total; the remainder is one "all other carriers" row	`claims.carrier_name`, `claims.recoverable_cents`, `claims.pending_review_cents`, `claims.created_at`
L2	Leakage by finding type	Over the **current packet only** of each claim created in the period: per `rule_id`, the count of findings, the sum of `amount_cents` where `needs_human_review = false` (identified) and where it is true (pending), split by `direction`	`packet_findings.rule_id`, `.direction`, `.amount_cents`, `.needs_human_review`, `evidence_packets` (current revision), `claims.created_at`
L3	Carrier by finding type	The L2 breakdown for one selected carrier key	as L1 and L2
L4	Leakage by lane	**Gap G3.** Not shown. A facility view from `extracted_fields.key = 'bol.facility'` is possible later but is not a lane	none

L2 must not double count. An edit copies every finding into the new revision, so summing findings across all revisions would multiply amounts by the number of edits. Only the current packet counts.

Rule ids are shown with their human title from `packet_findings.title` (the most recent title per `rule_id`), for example a linehaul rate above the rate confirmation, unauthorised accessorials or detention beyond free time.

4.5 Approval throughput

ID	Metric	Definition	Source
T1	Decisions per week	Count of `approvals` rows per ISO week in the period, by `action` (APPROVE, REJECT, SEND_READY, EDIT)	`approvals.action`, `approvals.created_at`
T2	Time to first decision	Median and 90th percentile, in hours, from a packet's `created_at` to the first APPROVE or REJECT row on that packet revision, for decisions made in the period	`evidence_packets.created_at`, `approvals.packet_id`, `approvals.packet_revision`, `approvals.created_at`
T3	Approval to send-ready	Median and 90th percentile, in hours, from the APPROVE row to the SEND_READY row on the same packet revision	`approvals`
T4	Review queue	Count and `recoverable_cents` sum of claims in `PENDING_REVIEW` now, and how many of them have been waiting more than 7 days (S2 entry time)	`claims`, as S2
T5	Reject rate	REJECT divided by (APPROVE + REJECT) in the period, shown with both counts; omitted below 5 decisions	`approvals.action`

The page shows team-level numbers only. It does not rank individual reviewers, even though `approvals.actor_id` would allow it; per-person productivity views are open question Q3.

5. Routes and implementation constraints

All routes are new, read-only and follow the existing conventions (`defineRoute`, Zod schemas in a new `packages/shared/src/insights-dto.ts`, strict outgoing schemas, rate group `intelligence`).

Method	Path	Access	Query	Returns
GET	`/api/v1/insights/summary`	`insights:read`	`from`, `to` (ISO dates), `kind?`, `perspective?`	M1-M7, S1, T4
GET	`/api/v1/insights/stages`	`insights:read`	`kind?`, `perspective?`	S1, S2, S3, S4 (S3 and S4 also take `from`, `to`)
GET	`/api/v1/insights/deadlines`	`insights:read`	`kind?`, `perspective?`	D1, D2
GET	`/api/v1/insights/leakage`	`insights:read`	`from`, `to`, `kind?`, `perspective?`, `by` (`carrier` or `rule`), `carrier?`, `limit?`	L1, L2, L3
GET	`/api/v1/insights/throughput`	`insights:read`	`from`, `to`, `kind?`, `perspective?`	T1, T2, T3, T5

Web: a new route `/insights` in `web/src/app-root.tsx` and a feature folder `web/src/features/insights/`, shown in the navigation only when the caller holds `insights:read`. Copy for every money label and the outcome disclaimer lives in `packages/shared/src/honest-copy.ts`.

Constraints:

- No new table, column, migration, trigger or tenant model. The page reads existing data only.
- Every query runs through `ctx.db` (tenant transaction); no system-mode access.
- Reads are not audited, matching `GET /api/v1/claims`. Exports of these numbers, if added later, are audited like the existing export routes.
- Query cost: periods are capped at 366 days and `limit` at 50. Existing indexes cover `claims (tenant_id, status)`, `(tenant_id, created_at)` and `approvals (tenant_id, claim_id, created_at)`. `claims.carrier_name` has no index; the planner decides whether L1 needs one after measuring on the seeded data set.

6. Gaps: what executives will ask for that the data model cannot answer

ID	Wanted metric	Why it is missing	Smallest data addition that would close it
G1	**Recovered dollars** (credited or paid by the carrier), recovery rate, time to payment	Nothing is recorded after `SEND_READY` (ADR 0007)	A tenant table `claim_outcomes` (claim id, sent at, channel, carrier response, credited cents, paid cents, closed at, evidence document), append-only, with RLS and audit, entered by a person
G2	Freight dispute deadlines	No contract or tariff dispute window is stored for truck freight	A per-carrier or per-contract `dispute_window_days` on a new tenant table, or a deadline field on the claim set at analysis
G3	Leakage by lane	Claims and extracted fields have no origin or destination	Two extracted keys (`origin`, `destination`, normalised to city and state or UN/LOCODE) and two claim columns
G4	Carrier win rate and response time	Depends on G1	G1
G5	Cost of recovery and ROI against the scoup.ai fee	No billing, plan or fee data in the product	A billing integration; out of scope for the product database
G6	Breakdown by business unit, site or team	No Team model (assumption A4)	A Team model with claim ownership

ID	Wanted metric	Why it is missing	Smallest data addition that would close it
G7	Exact point-in-time history of the open backlog	Claim amounts are current values; status history is reconstructable from `approvals`, amounts are not	Nightly snapshot rows, or accept reconstruction from status history only
G9	Overcharge as a share of audited freight spend	Claims hold only the disputed amounts; the invoice total sits in `extracted_fields` (`invoice.total`) of linked documents and is not a claim column, and invoices with no finding never become claims	A claim column for the audited invoice total set at analysis, plus a record for invoices audited with no finding
G10	Invoice payment due dates (short-pay before paying)	A shipper's leverage is to short-pay before the due date; freight invoices carry no due date in the extraction vocabulary	An `invoice.due_date` extracted field and claim column
G8	Benchmarks against other customers	Deliberately absent: tenants never see other tenants' data (ADR 0002)	Not proposed

Until G1 exists, the page must not show any figure called "recovered", "saved" or "won".

7. Acceptance criteria

Each criterion is written to become one or more tests. "Seed" means a synthetic fixture created by the test. Times use a fixed clock `now`.

Access and isolation

- **AC-ED-01** Given a user with role OWNER, ADMIN, MANAGER or VIEWER, when they call each of the five `/api/v1/insights/*` routes with valid parameters, then each answers 200.
- **AC-ED-02** Given a user with role REVIEWER or ANALYST, when they call any insights route, then it answers 403 and an `authz.denied` audit event is written (until Q1 changes the matrix).
- **AC-ED-03** Given a PLATFORM_DEV or SUPER_ADMIN user, when they call any insights route, then it answers 403.
- **AC-ED-04** Given a valid tenant API key with all three scopes, when it calls any insights route, then it answers 403.
- **AC-ED-05** Given no credentials, when any insights route is called, then it answers 401.
- **AC-ED-06** Given tenant A and tenant B each seeded with claims, when a tenant A user calls every insights route, then no response number changes when tenant B's amounts are multiplied by 1,000 and re-seeded.
- **AC-ED-07** `PERMISSION_MATRIX` grants `insights:read` to exactly OWNER, ADMIN, MANAGER and VIEWER, and the RBAC route-by-role test covers the five new routes.
- **AC-ED-08** Given the web app signed in as REVIEWER, ANALYST or a platform role, then no navigation item links to `/insights`, and opening `/insights` directly shows the existing forbidden state.

Validation

- **AC-ED-09** When `from` is not earlier than `to`, or the period exceeds 366 days, or `limit` exceeds 50, or `kind` or `by` has an unknown value, then the route answers 400 `validation_error` and lists only field paths and codes.

Money

- **AC-ED-10** Given seeded claims with known `recoverable_cents` in each of the five statuses, then M1 equals the sum over `AWAITING_ANALYSIS`, `PENDING_REVIEW` and `APPROVED` only.
- **AC-ED-11** Given a claim with `pending_review_cents > 0`, then that amount appears in M2 and in no other money metric.
- **AC-ED-12** Given SEND_READY approvals at exactly `from` and at exactly `to`, then M4 includes the first and excludes the second.

- **AC-ED-13** Given audited D&D invoices where one has a null `total_due_cents`, then M7 is omitted and the response states one excluded invoice; with all totals present, M7 equals the linked claims' identified sum divided by the invoices' total due.
- **AC-ED-14** Given `kind=OCEAN_DD`, then every metric counts only claims with `kind = 'OCEAN_DD'`; with no `kind`, both kinds count. Given `perspective=SHIPPER`, only `SHIPPER` claims count; the web page sends `perspective=SHIPPER` on first load and shows a control to change it.

Stages and deadlines

- **AC-ED-15** S1 returns all five statuses, including statuses with zero claims (count 0, sum 0).
- **AC-ED-16** Given a claim that moved to `APPROVED` 10 days before `now`, then S2 places it in the 8-30 day bucket of `APPROVED`; given an `AWAITING_ANALYSIS` claim created 2 days before `now`, then S2 uses `claims.created_at` and places it in 0-7.
- **AC-ED-17** Given open `OCEAN_DD` claims with `invoice_date` 25 and 31 days before `now`, then D1 places the first in 0-7 (5 days remaining) and the second in "already past"; a claim with a null `invoice_date` is counted only under "date missing".
- **AC-ED-18** D2 counts only open `FREIGHT` claims, never `SEND_READY` or `REJECTED` ones, and the response and UI label it "invoice age", never "deadline".
- **AC-ED-19** The D1 label in the UI includes the words "regulatory minimum" and cites 46 CFR 541.8(a).

Leakage

- **AC-ED-20** For L1, the top-N rows plus the "all other carriers" row sum exactly to the tenant total of `recoverable_cents + pending_review_cents` for the period.
- **AC-ED-21** Given claims with carrier names `"ACME Freight "` and `"acme freight"`, then L1 shows one group.
- **AC-ED-22** Given a claim whose packet was edited twice (three revisions, identical findings), then L2 counts each of its findings exactly once.
- **AC-ED-23** L2 splits each rule's amount into identified (`needs_human_review = false`) and pending (`needs_human_review = true`). As a consistency check, the identified amount in each claim's own direction (OVERCHARGE for `SHIPPER`, UNDERBILLED for `CARRIER`) summed over the same claims equals the sum of their `claims.recoverable_cents`; any rule where the engines define the total differently is recorded by the planner as a named exception, not loosened.
- **AC-ED-24** No insights response contains a lane, origin or destination field, and the UI shows G3 as "not tracked yet".

Throughput

- **AC-ED-25** T1 buckets decisions by ISO week in UTC and reports all four actions, with zero-count weeks present inside the period.
- **AC-ED-26** T2 measures from the decided revision's `created_at`; given an edit that creates revision 2, the clock for revision 2 starts at revision 2's `created_at`, not revision 1's.
- **AC-ED-27** T5 is omitted when the period has fewer than 5 APPROVE plus REJECT decisions.
- **AC-ED-28** No insights response or UI view lists individual users or `actor_id` values.

Honesty, determinism and safety

- **AC-ED-29** Every money figure in the rendered page carries one of the labels "identified", "pending review", "approved" or "ready to demand"; the words "recovered", "saved" and "won" appear only in the fixed disclaimer that says recovery outcomes are not tracked yet.
- **AC-ED-30** Given the same seeded data and the same `now`, two calls to each route return byte-identical bodies.

- **AC-ED-31** Calling all five routes changes no row in any table and appends no audit event other than `authz.denied` for refused callers.
- **AC-ED-32** The change adds no migration folder and leaves `api/prisma/schema.prisma` unchanged.
- **AC-ED-33** Exceeding `RATE_LIMIT_INTELLIGENCE_MAX` on the insights routes answers 429 with `Retry-After`.
- **AC-ED-34** A tenant with no claims gets 200 with zeros and empty lists from every route, and the UI shows empty states instead of errors.

Count: 34 acceptance criteria.

8. Open questions for the owner

- **Q1** Should REVIEWER and ANALYST also hold `insights:read`? They can already read every claim; the question is only about page clutter and who the page is for.
- **Q2** Should a read-only API-key scope (for example `insights.read`) exist so a customer can pull these numbers into their BI tool? That widens the machine surface (ADR 0006).
- **Q3** Do customers want per-reviewer throughput? It is computable from `approvals.actor_id` but invites surveillance concerns; this spec leaves it out.
- **Q4** Should G1 (`claim_outcomes`) be built before this page ships, so the first executive view can show real recovered money? Without it the page answers "found and approved", not "got back".
- **Q5** Is the 46 CFR 541.8(a) 30-day minimum the right default for D1, or should D1 wait for tariff-specific windows from `dd_tariffs.data`?